UNIVERSITÀ DEGLI STUDI DI MILANO BICOCCA

Geometrical architectures: Convolution Neural Networks

Lecture 14

Course of:
Statistics and data analysis

Federico De Guio - Martino Borsato

Context

- Will see **2** examples of geometrical architectures:
  - **Convolutional Neural Networks** (CNNs)
  - **Graph Neural Networks** (GNNs)

- TODAY:
  - Transformations on **digital images** (quick intro)
  - **CNN** for classification

Quick intro to **digital images**

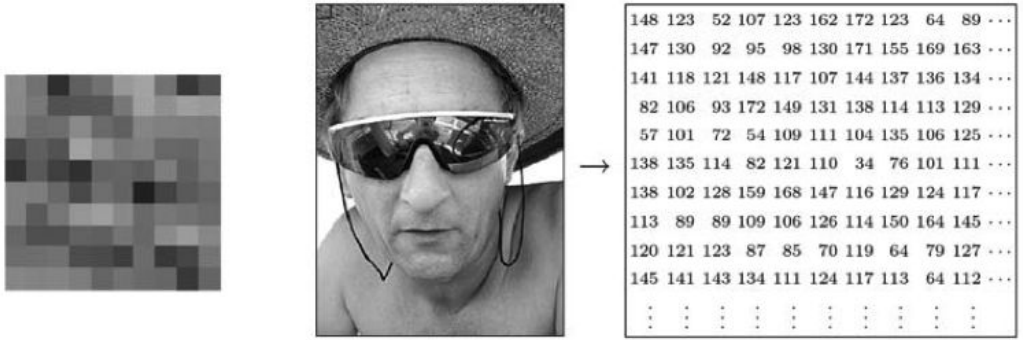
What is a **digital image**?

- 2-dimensional matrix of intensity (gray or color) values

Set of Intensity values

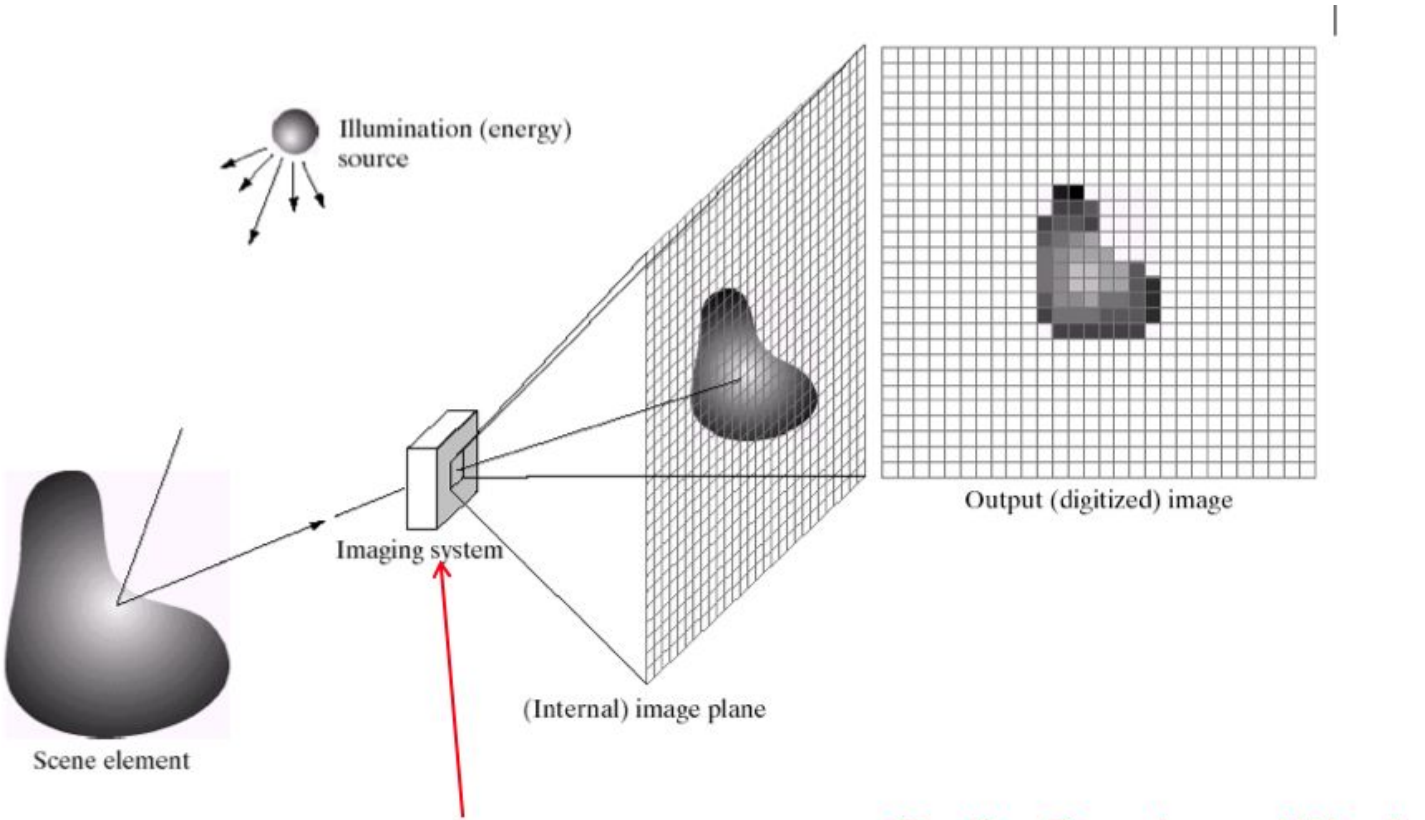
Image coordinates are integers

$$I(u, v) \in \mathbb{P} \quad \text{and} \quad u, v \in \mathbb{N}.$$



$F(x, y)$ $I(u, v)$

Imaging system

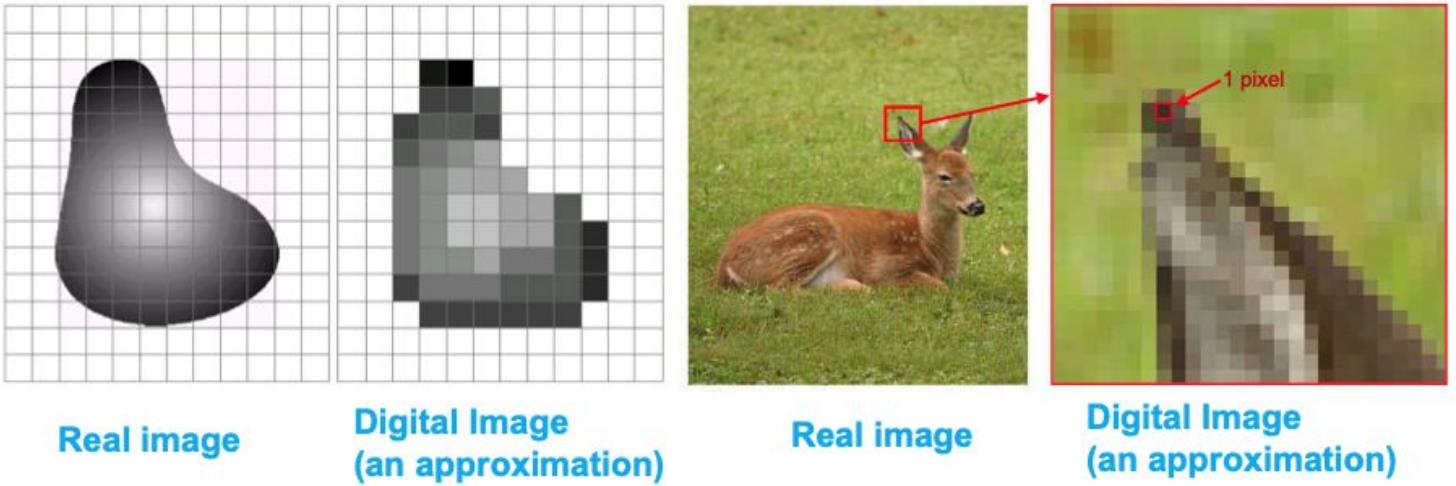


Example: a camera
Converts light to image

Credits: Gonzales and Woods

Image **digitization**

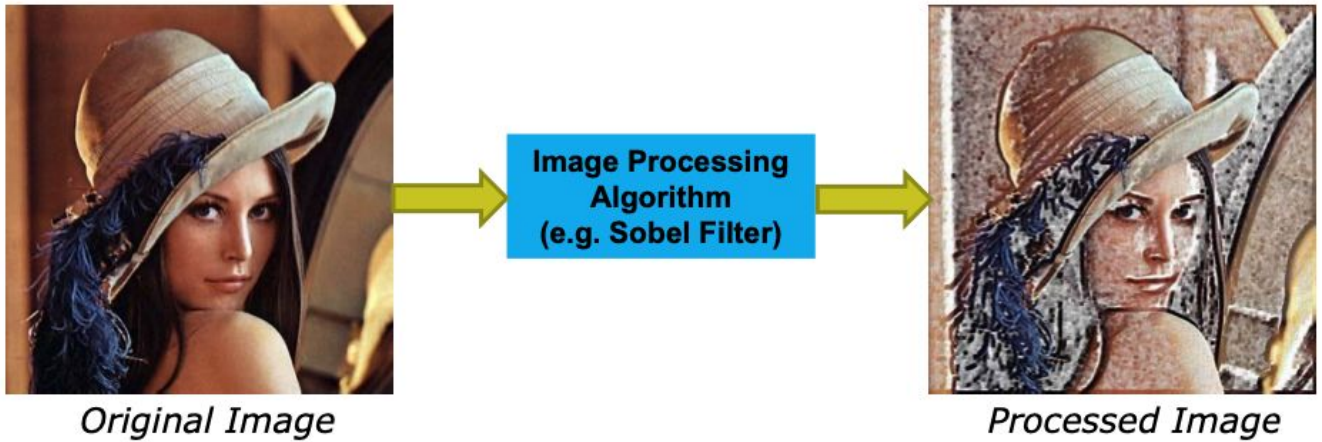
- **Digitization** causes a digital image to become an **approximation** of a real scene



- 1 value per pixel: B&W or Grayscale
- 3 values per pixel: Color image (RGB)

What is image processing?

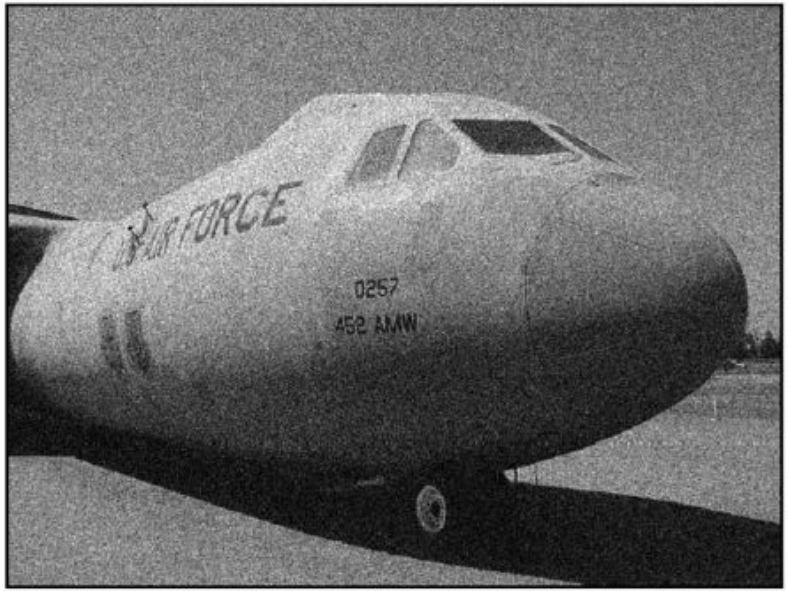
- Algorithms that alter an input image to create new image
  - **The input is an image, the output is an image**



- **Improve images for human or AI interpretation**
  - Image and features enhancement

An example: **noise removal**

Noisy Image



Denoised Image



An example: **contrast adjustment**



Low Contrast



Original Contrast



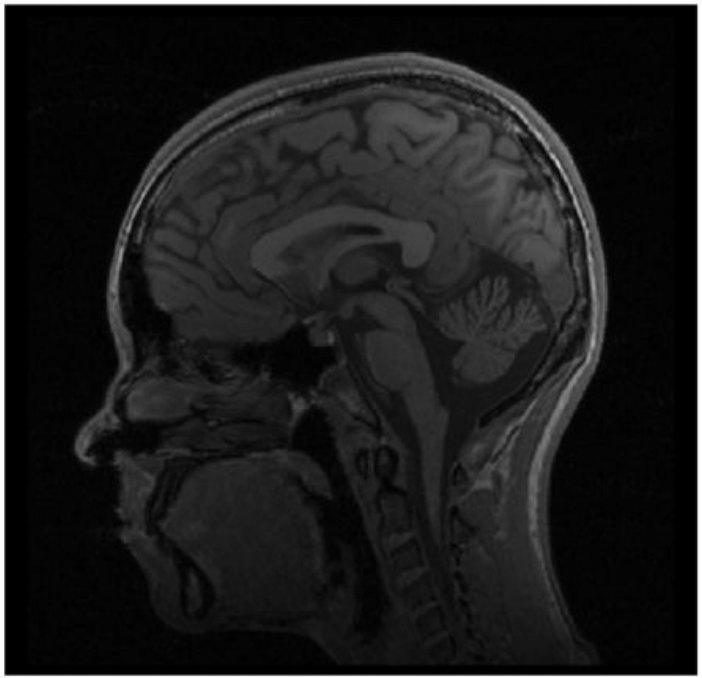
High Contrast

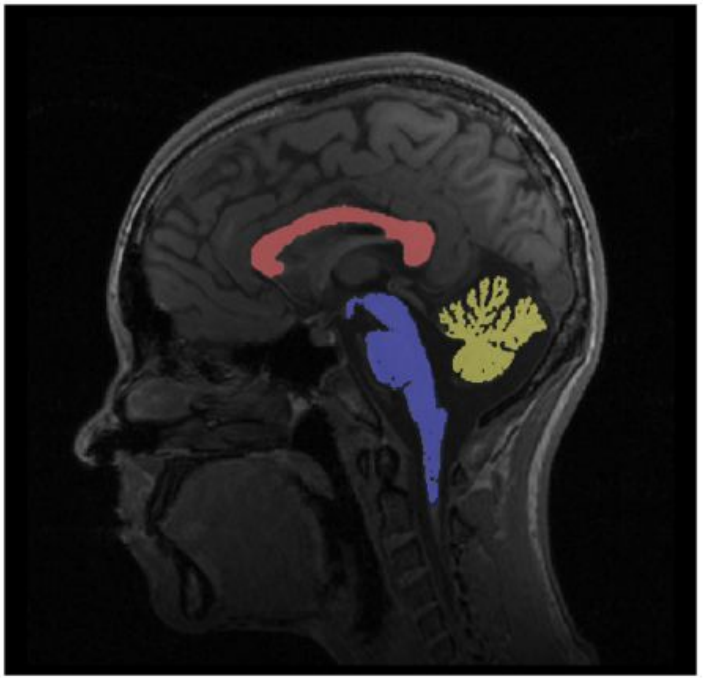
An example: **edge detection**





An example: region detection and **segmentation**





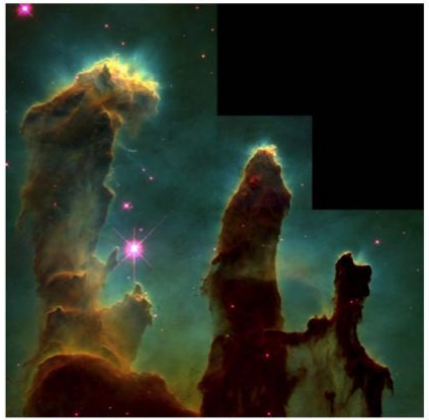
Application of image processing

- **Applications are countless:**
  - Biology
  - Astronomy
  - Medicine
  - Security and biometric
  - Precision agriculture
  - Satellite imagery and terrain classification
  - Meteorology
  - Art
  - …

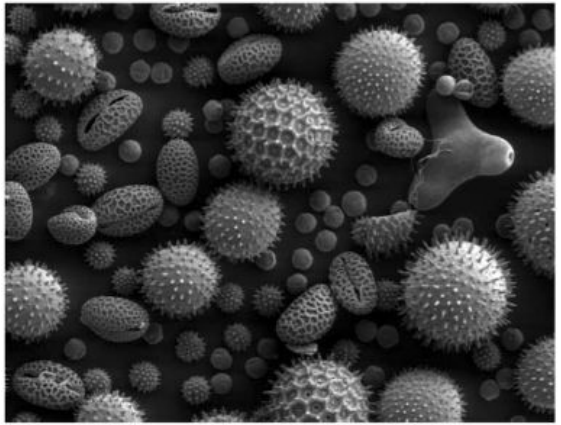


Credit: NASA





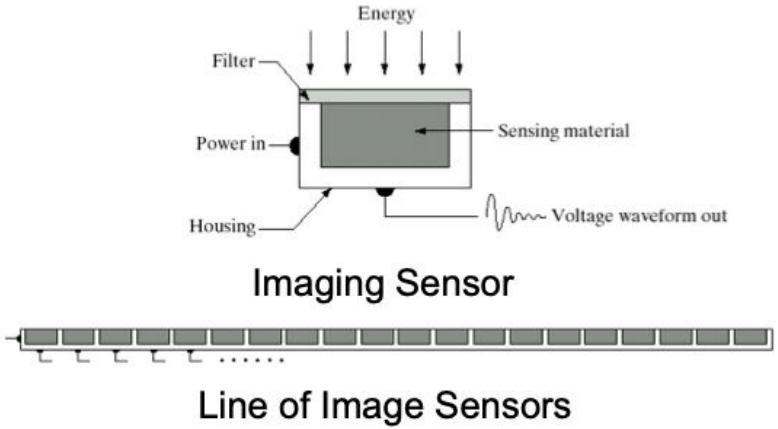
Credit: NASA, Jeff Hester, and Paul Scowen (Arizona State)
More info here

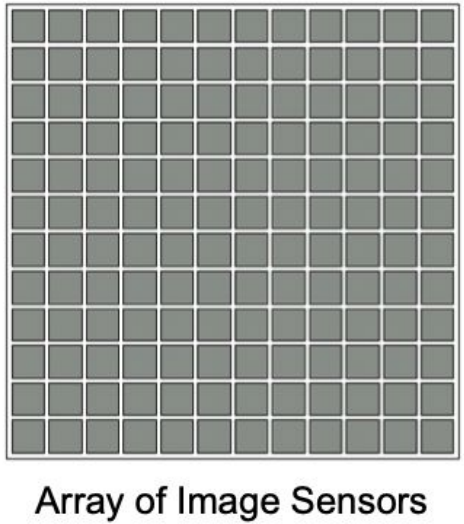


Credit: Dartmouth Electron Microscopy Facility

Image acquisition

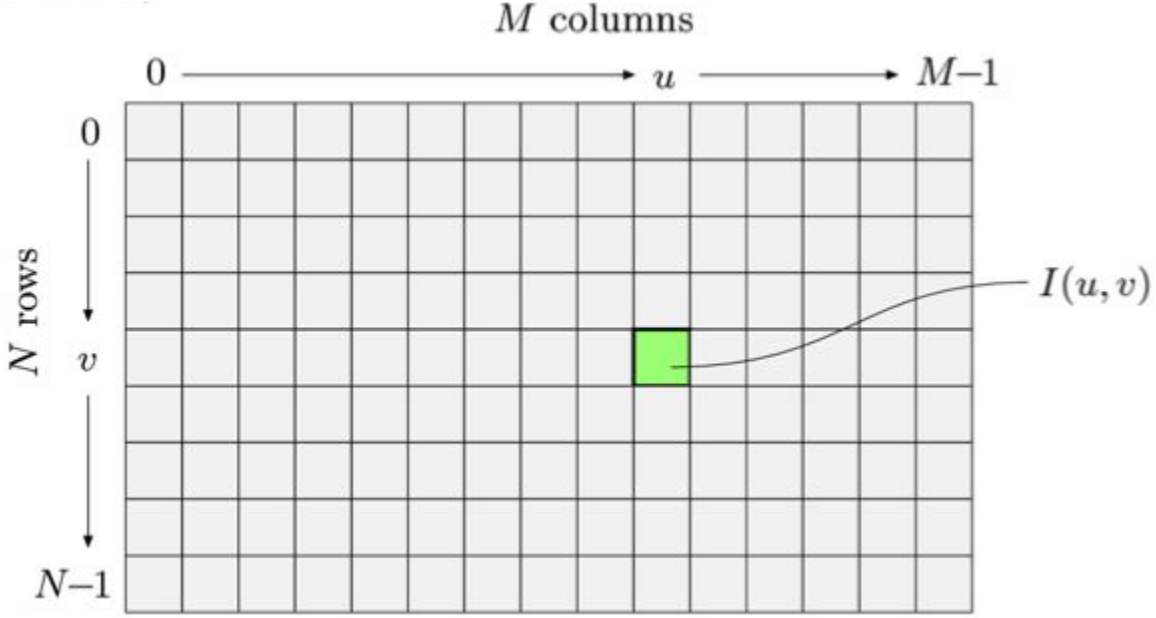
- The incoming energy (**light**) hits a sensor material which is **responsive to that type of energy**
- Collections of sensors are arranged to capture images
  - **Record image values at discrete x, y (e.g. 10 MegaPixel)**
  - **Discrete set of energy levels (e.g. 8-bit grayscale)**





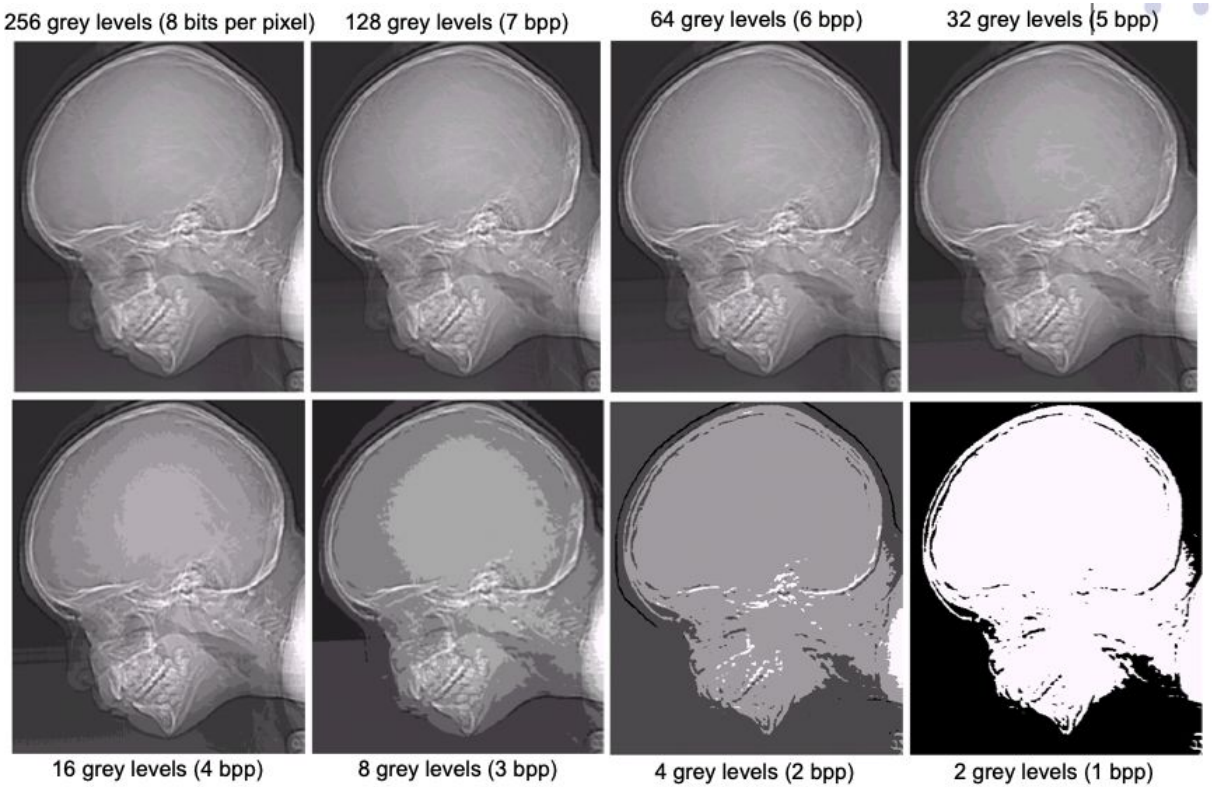
Representing images

- The image data structure is a 2D array of pixel values
- Pixel values are gray levels in the range 0-255 (or RGB colors)



Intensity level resolution

- Is the **number of intensity levels** used to represent the image

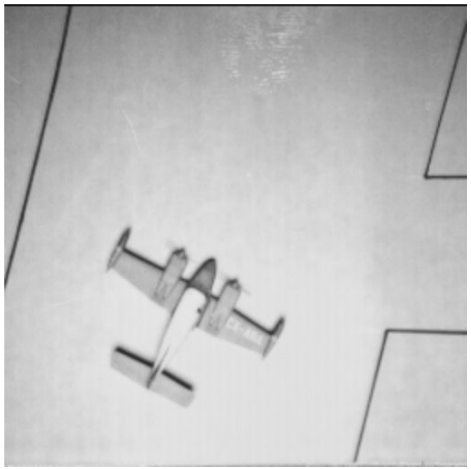


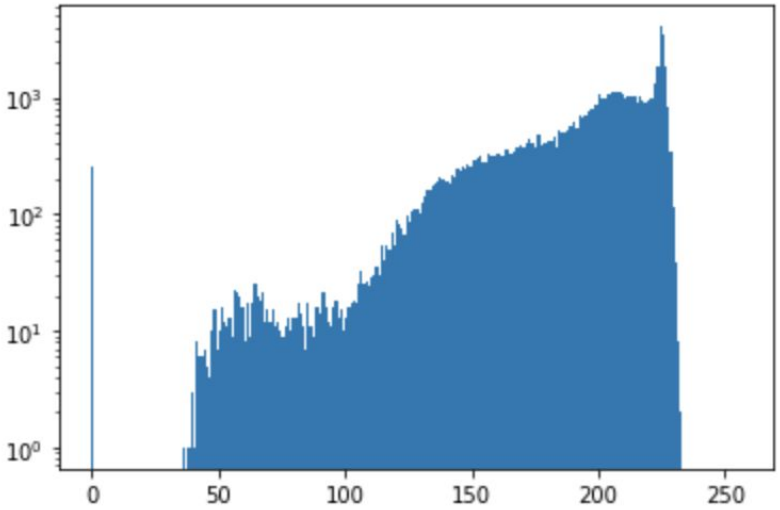
Histograms

for you to read autonomously

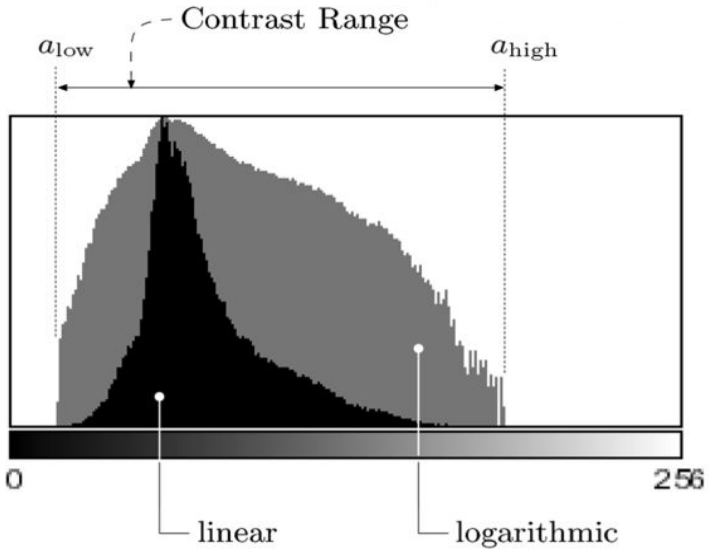
Histograms

- They show **how many times each intensity value occurs in the picture**





Histograms



Brightness

$$B(I) = \frac{1}{wh} \sum_{v=1}^{h} \sum_{u=1}^{w} I(u, v)$$

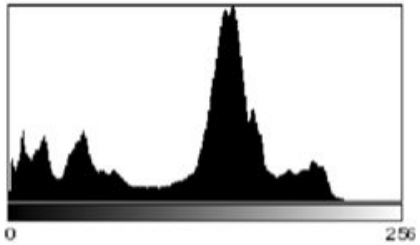
2. Divide by total number of pixels

1. Sum up all pixel intensities

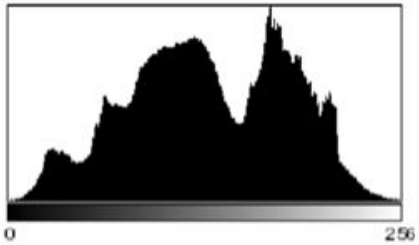
Exposure



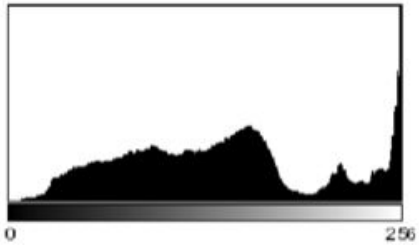
Image



(a)



(b)



(c)

Histogram

Underexposed

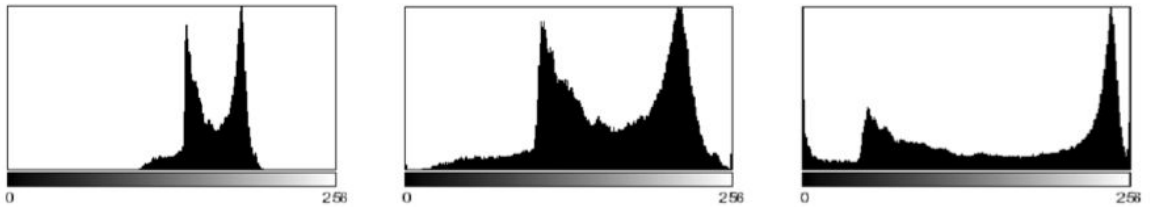
Properly Exposed

Overexposed

Contrast



Image



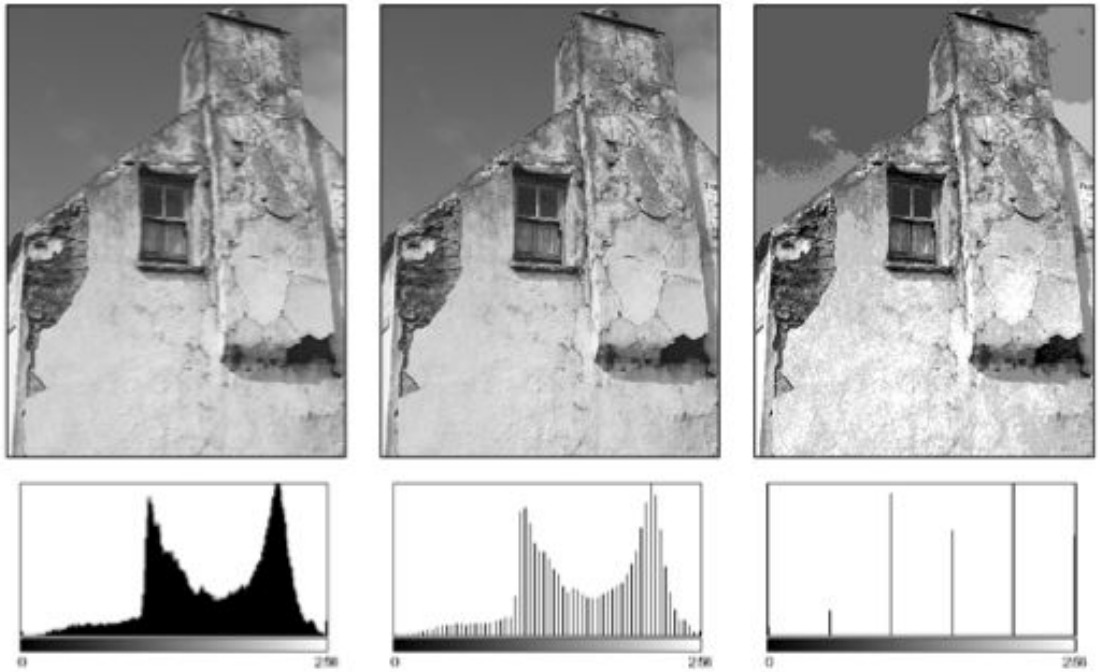
Histogram

(a) Low contrast

(b) Normal contrast

(c) High contrast

Dynamic range



(a)

High Dynamic Range

(b)

Low Dynamic Range (64 intensities)

(c)

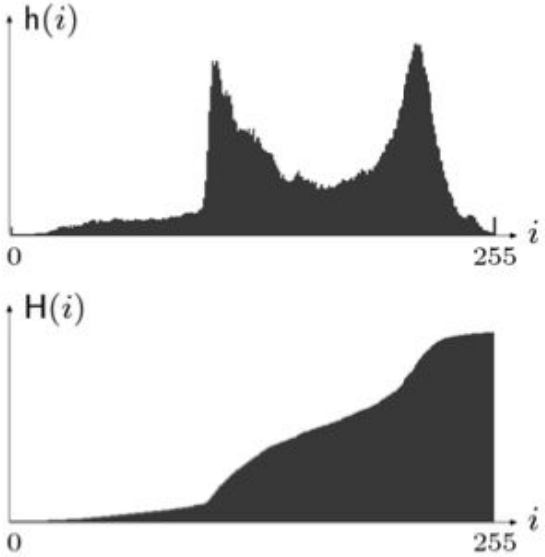
Extremely low Dynamic Range (6 intensity values)

Computing histograms

- Question for you: **How would you compute the histogram of an image?**
  - Range of the x axis?
  - Number of bins?
  - What about color images?

- Sometimes the **cumulative distribution** gives useful information
  - E.g. for histograms equalization





Point operations

What do they do?

- Change the pixel's intensity value according to some function
  - Homogeneous operations

- New pixel intensity **doesn't depend on:**
  - The pixel location (x, y)
  - The pixel neighbours

- New pixel intensity **depends on:**
  - The pixel previous intensity
  - The mapping function

$$a' \leftarrow f(a)$$
$$I'(u,v) \leftarrow f\big(I(u,v)\big)$$

Example: thresholding

- The input values below **threshold a_{th}** are set to **a_0**
- The input values below **threshold a_{th}** are set to **a_1**

$$f_{\text{threshold}}(a) = \begin{cases} a_0 & \text{for } a < a_{\text{th}} \\ a_1 & \text{for } a \geq a_{\text{th}} \end{cases}$$

- Converts the grayscale image to binary image



Original Image



Thresholded Image

Example: grey levels transformation - the power law

- There are many options. A commonly used transformation is the **power law**
- Map a narrow range of dark input values into a wider range of output values or vice versa

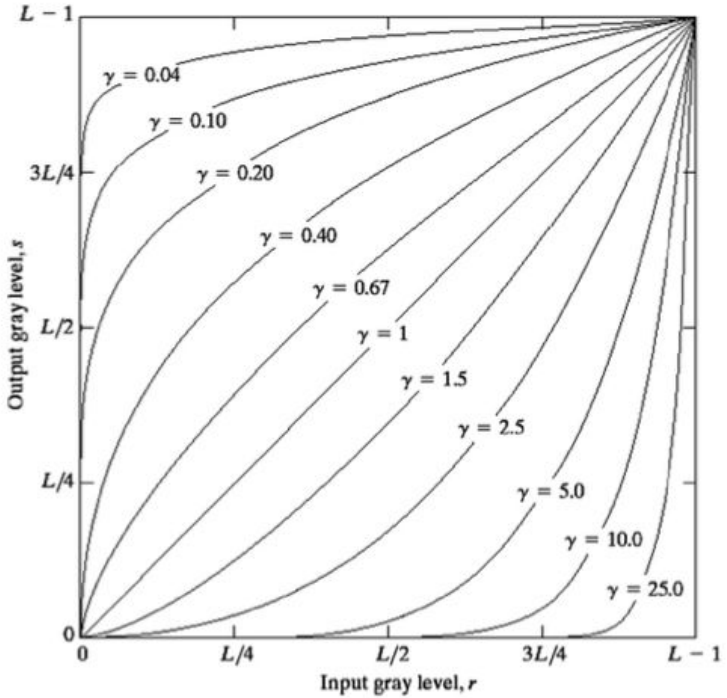
$$s = c * r^{\gamma}$$

Power

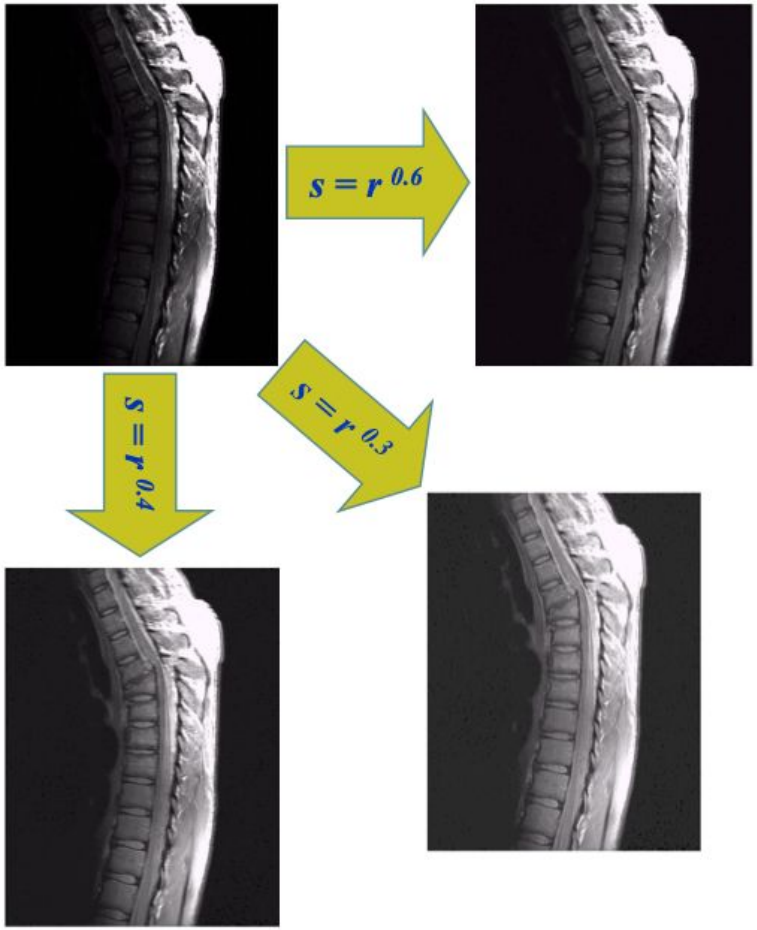
Old pixel value

New pixel value

Constant



Different power values highlight different details

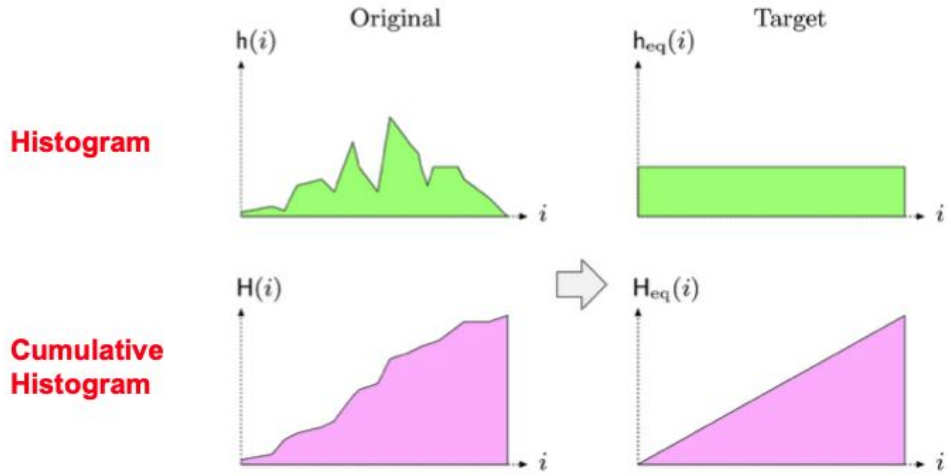


Example: histograms equalization

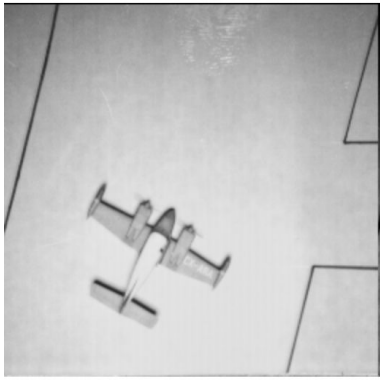
- Adjust two different images to make their histograms (intensity distributions) similar
- Apply a point operation that changes the histogram to a **uniform distribution** as much as possible

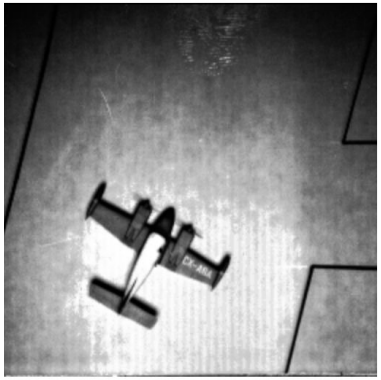
processed intensity → $s_k = T(r_k)$ ← input intensity

Intensity range
(e.g 0 – 255)



Example: linear histograms equalization

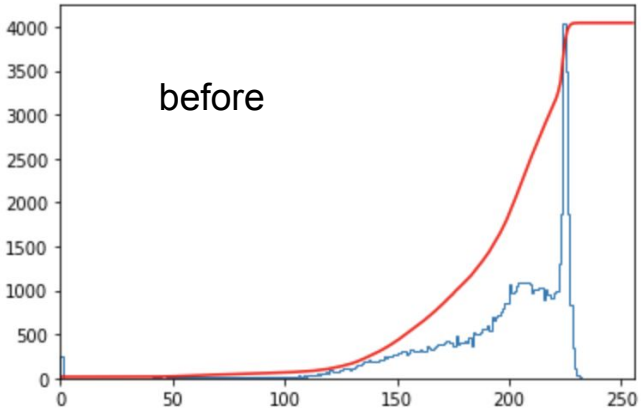


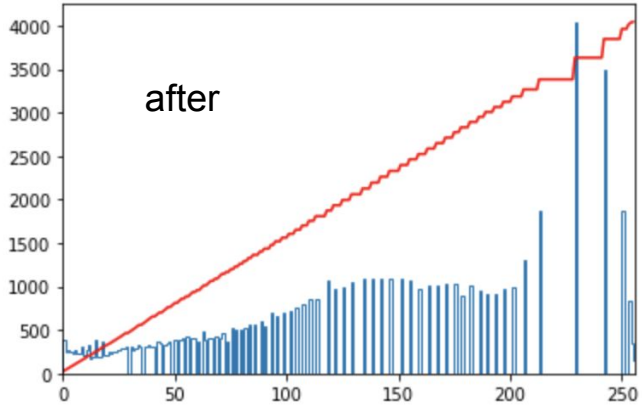


Point operation that returns Linear equalized value of *a*

$$f_{\text{eq}}(a) = \left\lfloor \mathsf{H}(a) \cdot \frac{K-1}{MN} \right\rfloor$$

Cumulative Histogram: Σ how many times intensity a occurs





Filters

What is a filter?

- Capabilities of point operations are limited
- **Filters: combine pixel's value + values of neighbors**



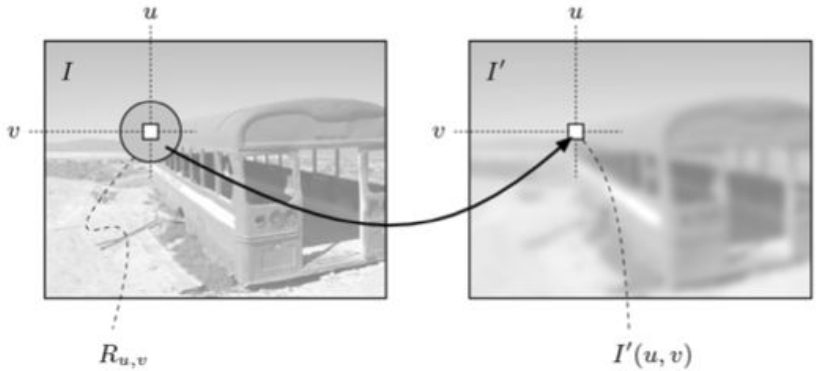


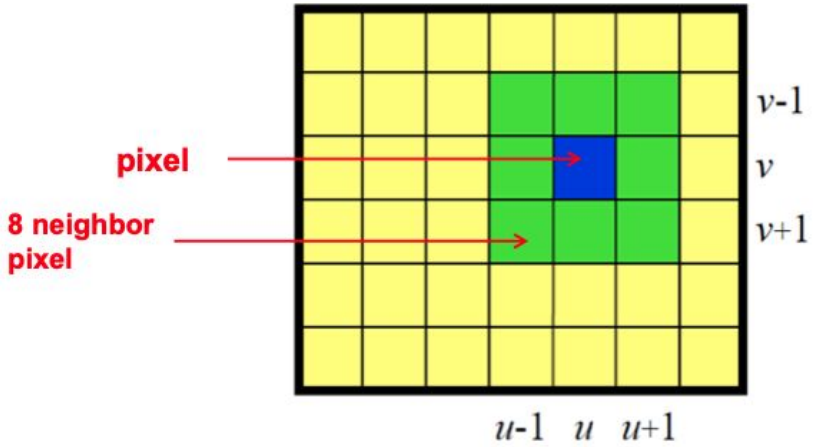
- **E.g. burring:** compute the average intensity of blocks of pixels

Average of a 3x3 matrix

- **Blurring:** Replace each pixel with the average intensity of pixel+neighbors

$$I'(u,v) \leftarrow \frac{p_0 + p_1 + p_2 + p_3 + p_4 + p_5 + p_6 + p_7 + p_8}{9}$$

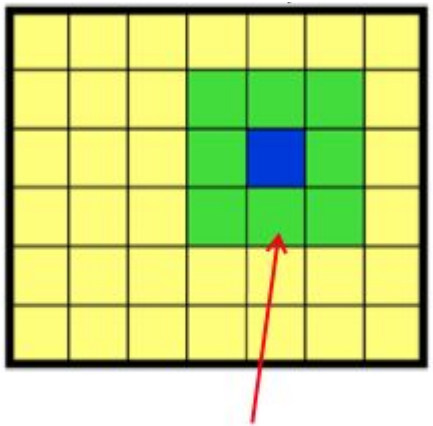




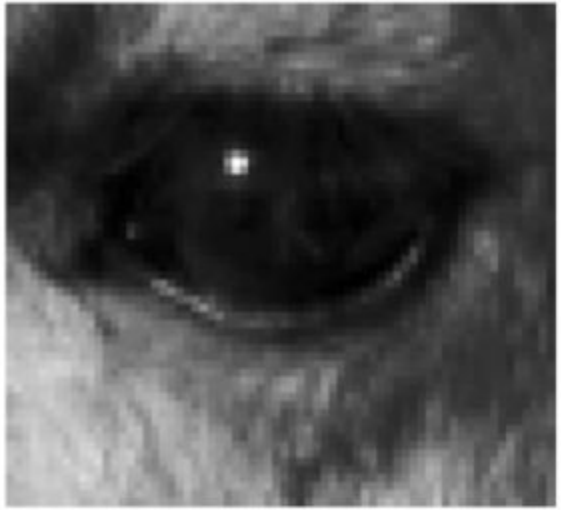
- **In general:** $I'(u,v) \leftarrow \frac{1}{9} \cdot \sum_{j=-1}^{1} \sum_{i=-1}^{1} I(u+i, v+j)$

Filters parameters

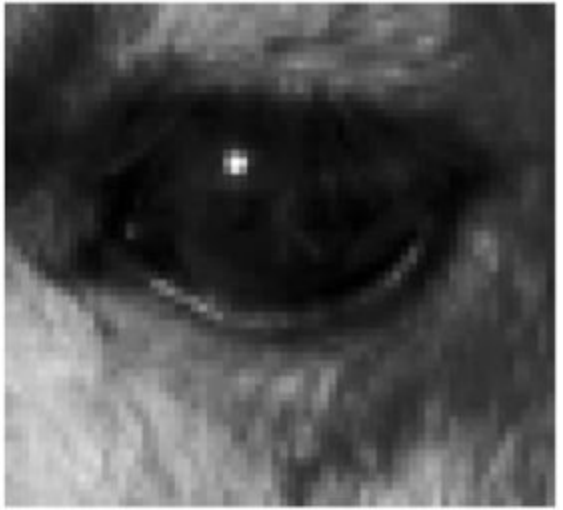
- **Filter size (size of neighborhood):** 3x3, 5x5, 7x7, ...,21x21,..
- **Filter shape:** not necessarily square. Can be rectangle, circle, etc
- **Filter weights:** May apply unequal weighting to different pixels
- **Filters function:** can be linear (a weighted summation) or nonlinear



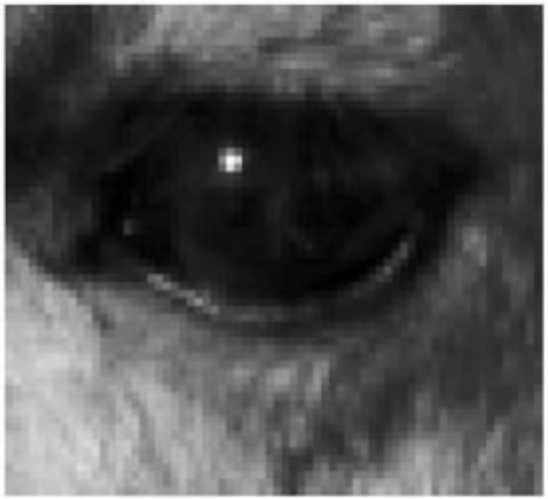
What does this filter do?



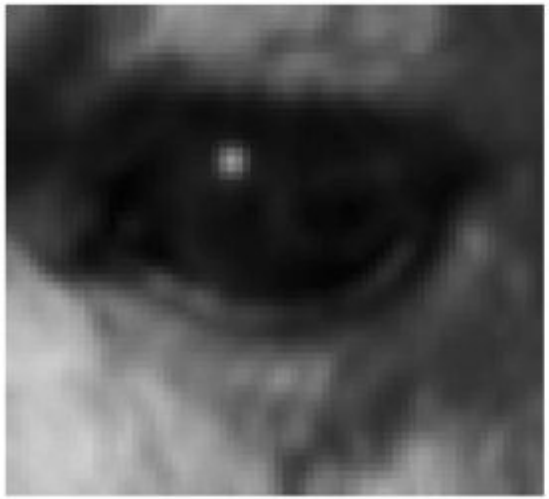
0	0	0
0	1	0
0	0	0



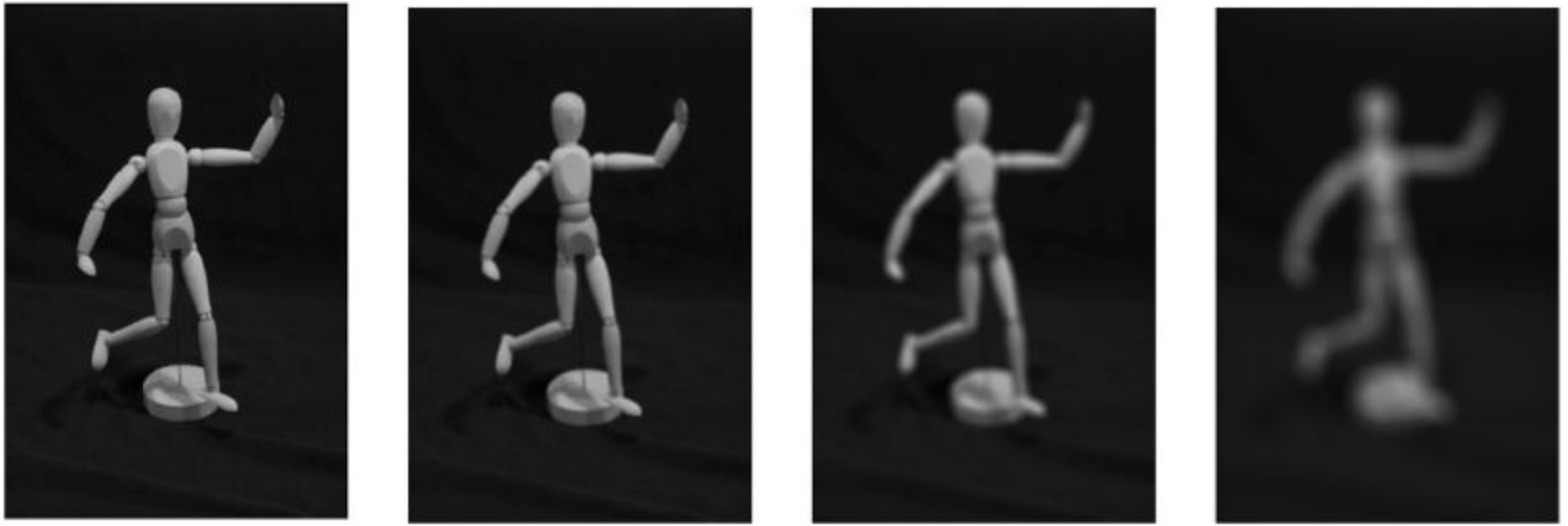
What does this filter do?



$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



Effect of filter size



Original 7×7 15×15 41×41

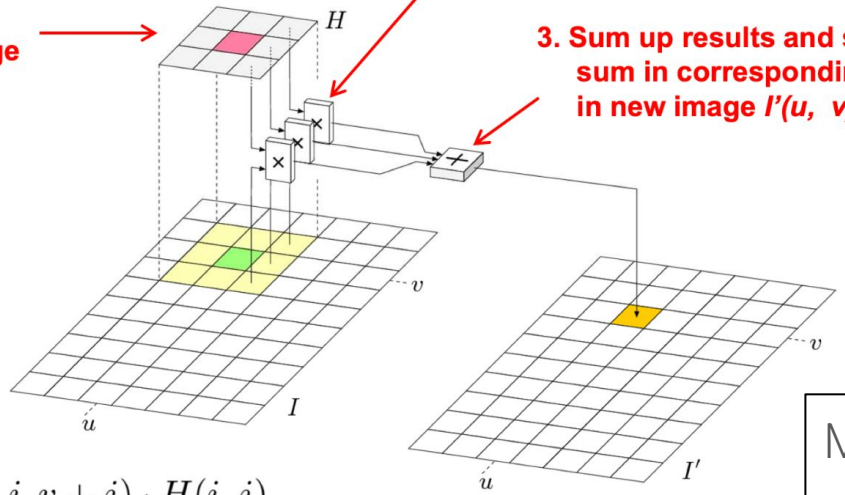
Applying linear filters: **convolution**

For each image position *I(u,v)*:

1. Move filter matrix *H* over image such that *H(0,0)* coincides with current image position *(u,v)*

2. Multiply all filter coefficients *H(i,j)* with corresponding pixel *I(u + i, v + j)*

3. Sum up results and store sum in corresponding position in new image *I'(u, v)*



Stated formally:

$$I'(u,v) \leftarrow \sum_{(i,j)\in R_H} I(u+i, v+j) \cdot H(i,j)$$

R_H is set of all pixels Covered by filter. For 3x3 filter, this is:

$$I'(u,v) \leftarrow \sum_{i=-1}^{i=1} \sum_{j=-1}^{j=1} I(u+i, v+j) \cdot H(i,j)$$

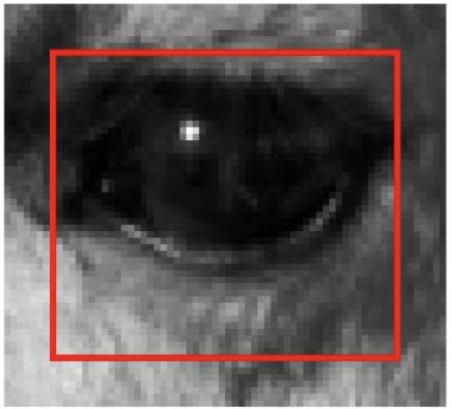
Many useful filters satisfy:

- **Linearity**
- **Translation invariance**

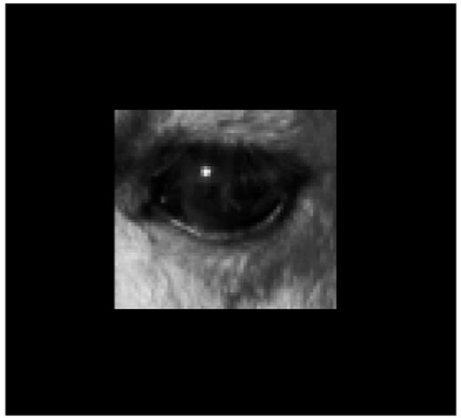
Theorem: any operation with these two properties is a convolution

How to handle the boundaries?

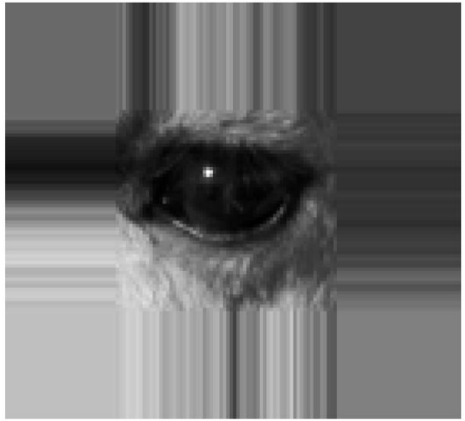
Crop



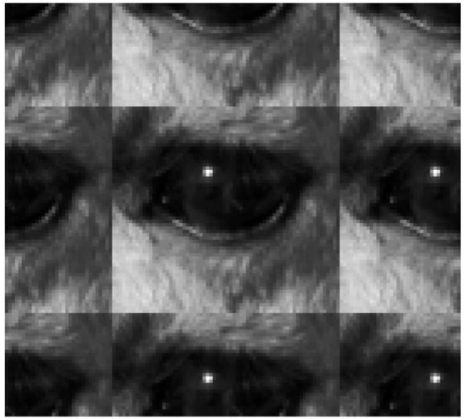
Pad



Extend



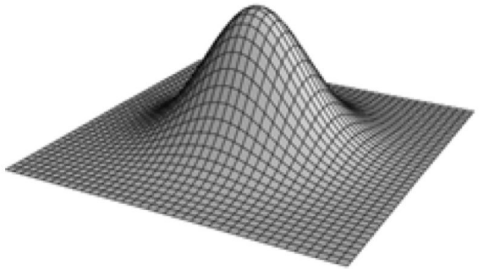
Wrap



Smoothing filters

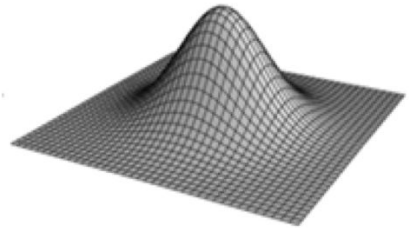
- More effective smoothing filters can be generated by associating different weights to the pixels in the neighbourhood
  - **Larger weight applied to the center**
  - **Normalization is preserved**

$$H(i,j) = \begin{bmatrix} 0.075 & 0.125 & 0.075 \\ 0.125 & \mathbf{0.2} & 0.125 \\ 0.075 & 0.125 & 0.075 \end{bmatrix}$$

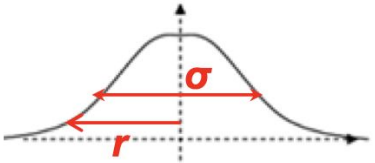


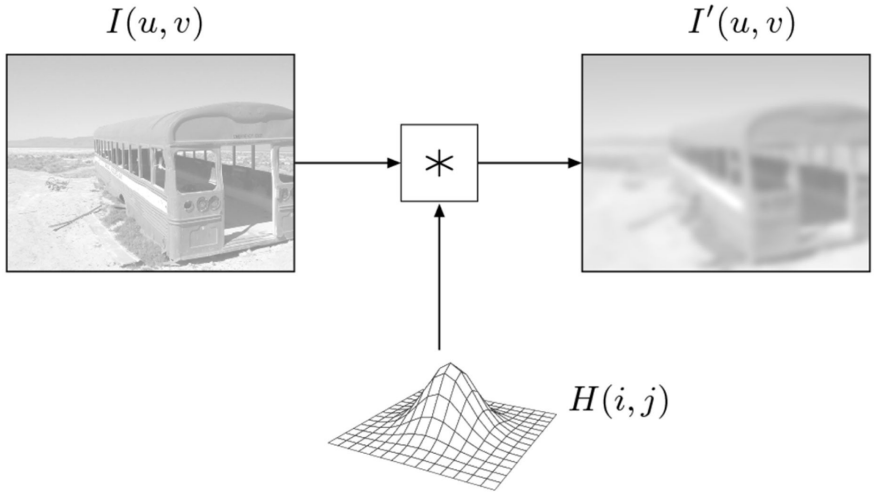
An example: the gaussian filter

$$G_\sigma(r) = e^{-\frac{r^2}{2\sigma^2}} \qquad \text{or} \qquad G_\sigma(x,y) = e^{-\frac{x^2+y^2}{2\sigma^2}}$$



- **σ is width** (standard deviation)
- **r is distance from center**





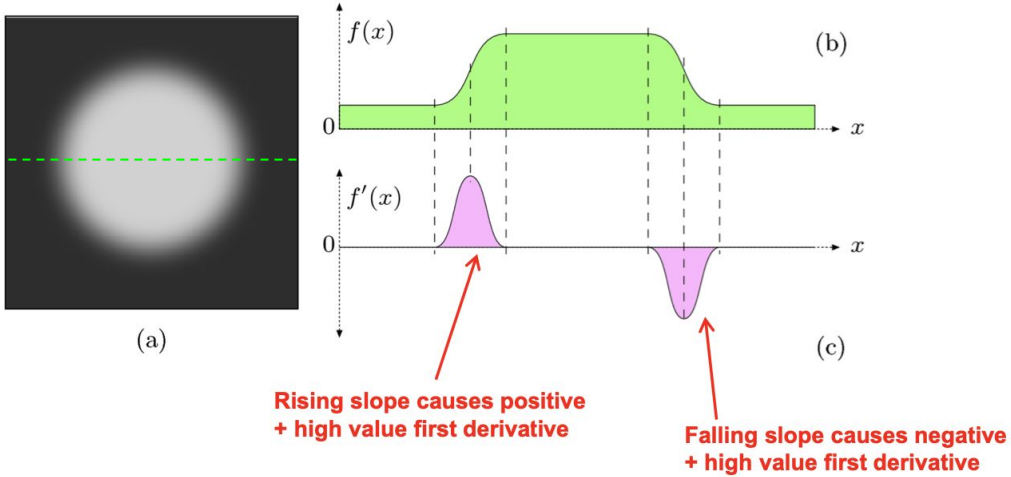
0	1	2	1	0
1	3	5	3	1
2	5	9	5	2
1	3	5	3	1
0	1	2	1	0

Gaussian filter

Other types of filters: edge detection

- Filters can be used to highlight features in the image
- For example **detect the edges as sharp changes in brightness**
  - Can occur in boundaries between objects or within the same object (e.g. zebras)
- Ideal edges are step functions. In real life the **step** is smooth and **can be detected by looking at the first derivative**

$$f'(x) = \frac{df}{dx}(x)$$

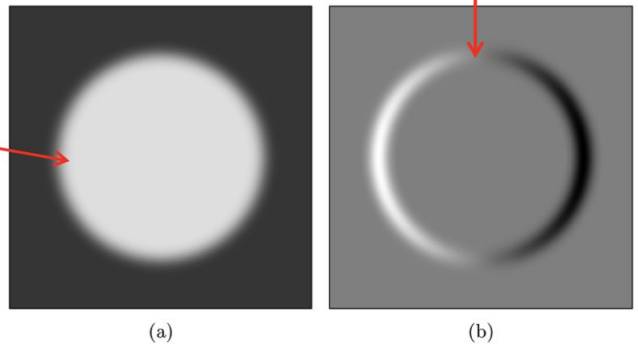


Derivative filters

Gradient slope in horizontal direction

$$H_x^D = \begin{bmatrix} -0.5 & \mathbf{0} & 0.5 \end{bmatrix} = 0.5 \cdot \begin{bmatrix} -1 & \mathbf{0} & 1 \end{bmatrix}$$

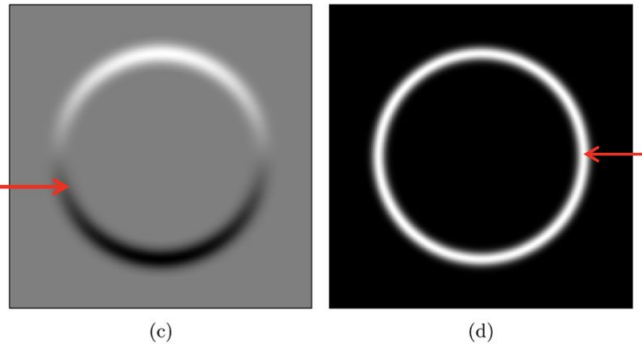
A synthetic image



(a) (b)

$$H_y^D = \begin{bmatrix} -0.5 \\ \mathbf{0} \\ 0.5 \end{bmatrix} = 0.5 \cdot \begin{bmatrix} -1 \\ \mathbf{0} \\ 1 \end{bmatrix}$$

Gradient slope in vertical direction



(c) (d)

Magnitude of gradient

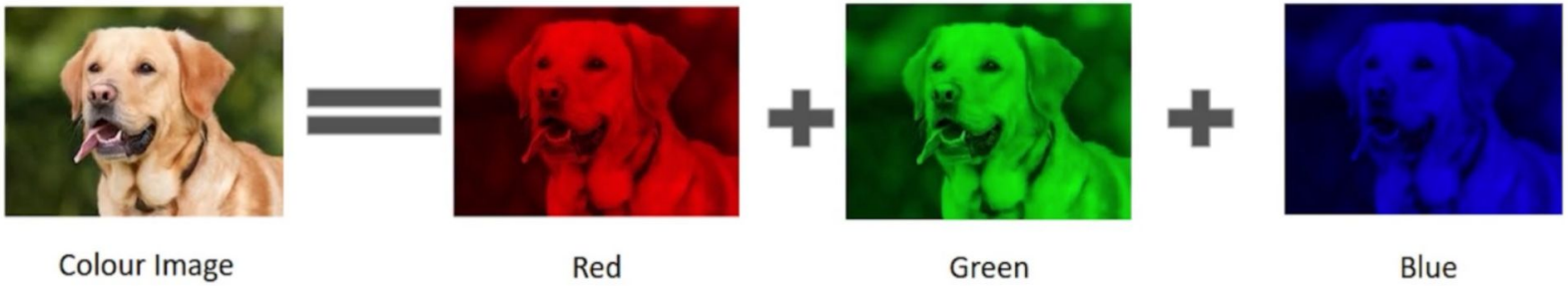
Where does the derivative filter shape come from?

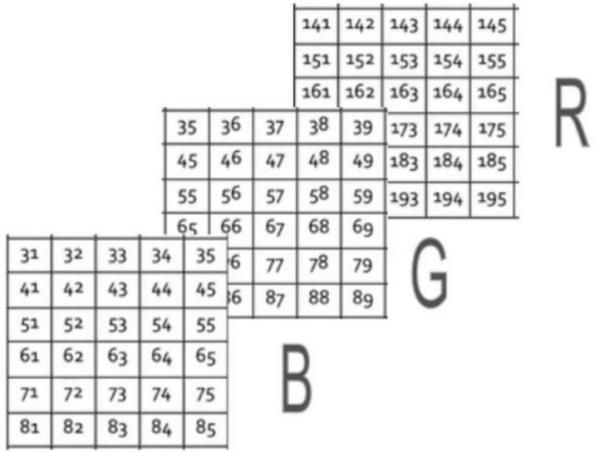
- The derivative is defined as: $f(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$
- Since we work with pixels, we could use the **discrete version** of it: $f(x) \approx \frac{f(x+1) - f(x-1)}{2}$
  - The distance is the pixel
  - Centered
  - Symmetric
- If I now consider the convolution operation between image and the 3x1 filter h(k) **[-0.5, 0, 0.5]** I get:

$$(f * h)(x) = \sum_{k=-1}^{1} f(x-k)h(k) \longrightarrow (f * h)(x) = f(x-(-1))h(-1) + f(x-0)h(0) + f(x-1)h(1)$$

$$\longrightarrow \quad (f * h)(x) = -0.5\, f(x+1) + 0.5\, f(x-1)$$

Natural extension: colored images



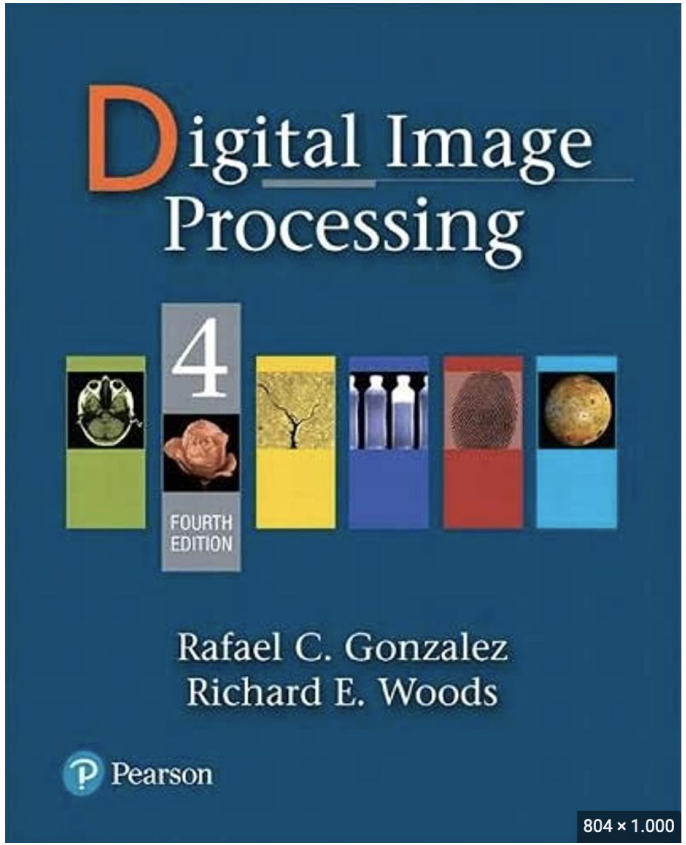


- Colored images are usually coded with the **RGB color model**: each pixel is associated to three numbers, corresponding to the **Red, Green and Blue intensity**

- An RGB image is therefore represented by a matrix (width)x(height)x3
- A greyscale image is (width)x(height)x1

Digital image processing

- A good reference is the book
  - **Digital image processing - Pearson (2018)**
  - **Gonzalez, Rafael C. - Woods, Richard E.**



Play with images with ***OpenCV***

- Pick an image among the ones here (suggestion: start with a grayscale image) and:
  - Show the original picture
  - Display its histogram and its cumulative distribution: comment the histogram
  - Transform the image from grayscale to B&W and show its histogram: comment the histogram
  - Equalize the image so that the full dynamic range is used, and show its histogram
  - Show the negative of the image
  - Apply a filter that enhances the edges
  - What features are highlighted?
  - Mirror, zoom and rotate the image (relevant for data augmentation)

- A basic example on how to use **openCV** can be found here

Convolutional Neural Networks

Our goal of today:

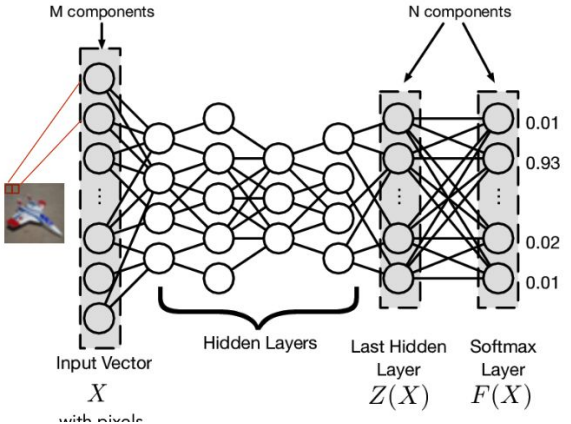
Classify images using ML

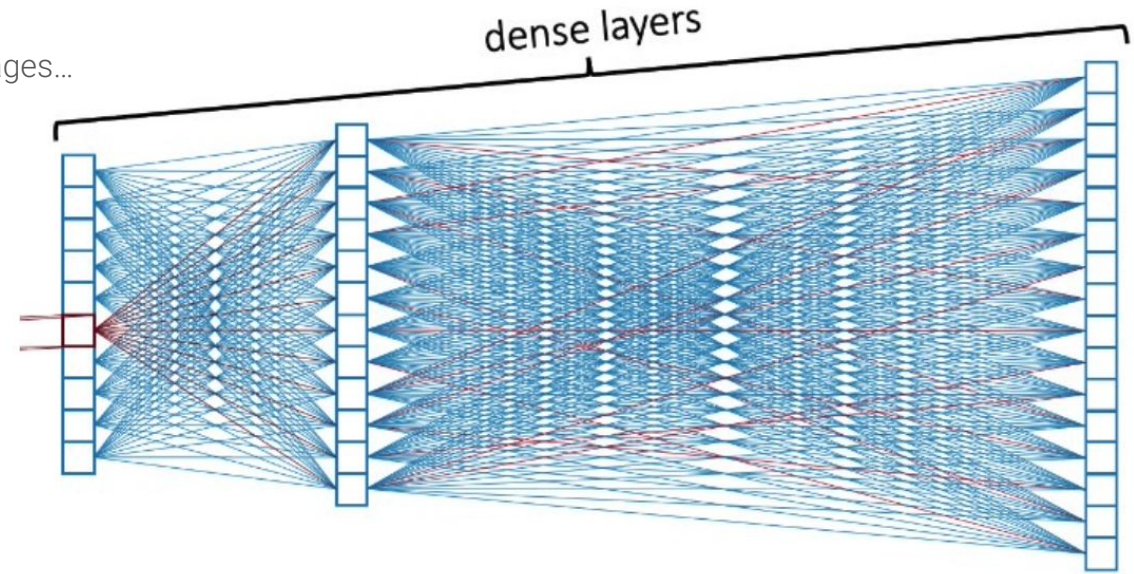
- James, cap.10.3
- Prince, cap.10

Images classification with DNN

- In principle **a FF DNN could be used to classify images**
  - Flatten the image into a 1D vector of pixels

- In practice, if we take a **64x64 grayscale** image:
  - 1D vector of 4096 values → **4096 nodes in the input layer**
  - if the (fully connected) inner layer has **500 nodes**, we will have 4096x500+4096= **2052096 weights** between the input and the hidden layer
  - Account for a factor 3 for colored images…





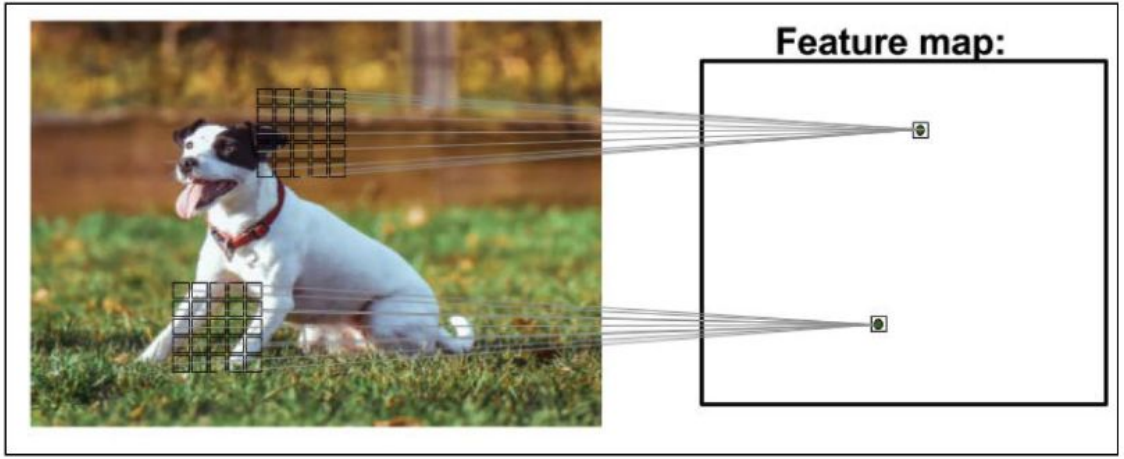
DOES NOT SCALE!!!

Convolutional Neural Networks (**CNN**)

- To flat an image into a 1D array is not the best way to model images
  - any **spatial relationship in the data is ignored**

- A CNN **maintains the spatial structure of the data**, and is better suited for finding spatial relationships in the image data

- **The idea behind:** use filters that automatically learns the most discriminants features in an image, such as edges, filled patterns, specific geometric forms and so on…

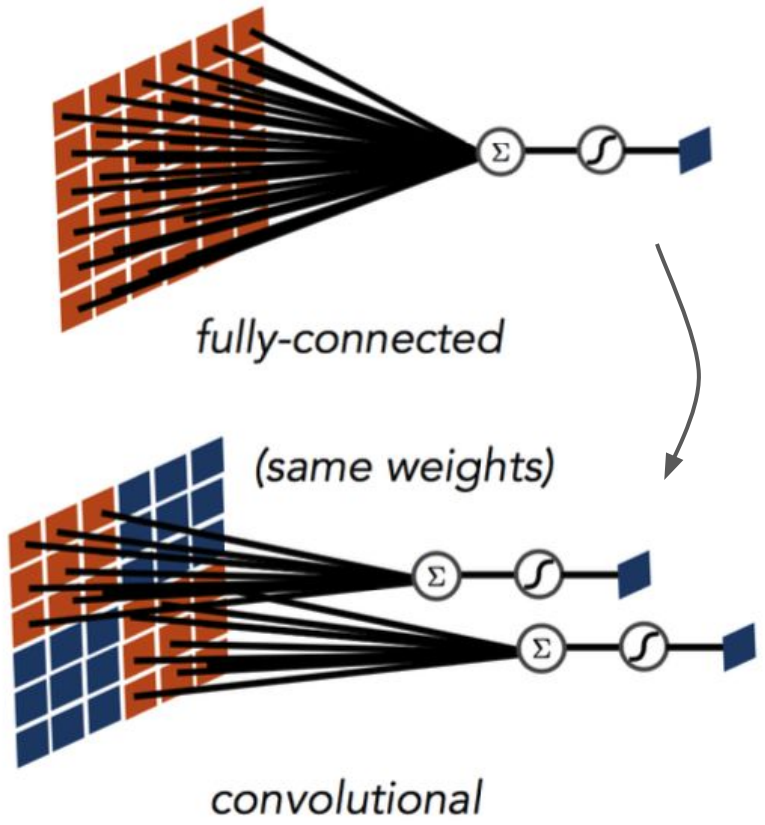


Convolutional Neural Networks (**CNN**)

2 main assumptions

- **Locality:**
  - nearby areas tend to contain stronger patterns
  - → inputs can be restricted to regions
- **Translation invariance:**
  - Parameters are shared across (x, y)
  - → same weights

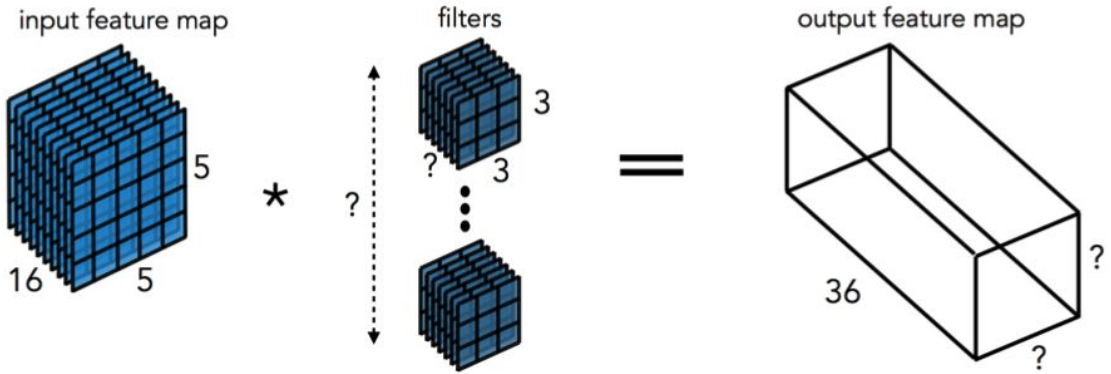
- **Advantages**:
  - Save a lot of parameters
  - fast, highly **parallelizable** on GPUs



The convolutional layer

- It is the core **building block of a Convolutional Network**

- The CONV layer's parameters consist of a **set of learnable filters**
- Each filter outputs an activation map → **Conv layer outputs a volume**



With: stride=1 and padding=0

1. How many filters are there?
36

2. What's the depth of each filter?
16

3. What's the output size?
3 x 3 x 36

The convolutional layer - the math behind

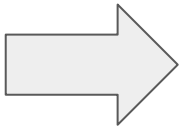
- A discrete **convolution between two vectors** with finite size **x** and **w** is mathematically defined as:

$$\boldsymbol{y} = \boldsymbol{x} * \boldsymbol{w} \rightarrow y[i] = \sum_{k=-\infty}^{+\infty} x[i-k]\, w[k]$$

- **w** is typically called the **filter or kernel**
- The index i runs through each element of the output vector y
- In machine learning applications **we always deal with finite feature vectors**

The convolutional layer - the math behind

- Let's assume that **x** and **w** have **n** and **m** elements respectively, where $m \leq n$:

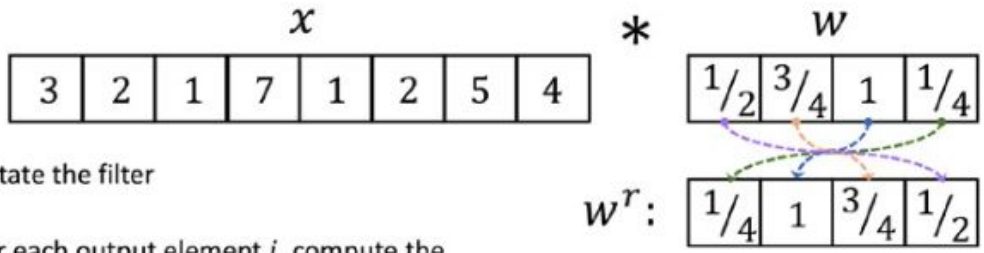


$$y = x * w \rightarrow y[i] = \sum_{k=0}^{k=m-1} x[i+m-k]w[k]$$

- It's important to notice that **x** and **w** are indexed in different directions in this summation
- Computing the sum with one index going in the reverse direction is equivalent to computing the sum with **both indices in the forward direction after flipping one of those vectors**
- This operation is repeated in a sliding window approach to get all the output elements

The convolutional layer - a practical example

x = [3 2 1 7 1 2 5 4], w = [½, ¾, 1, ¼]



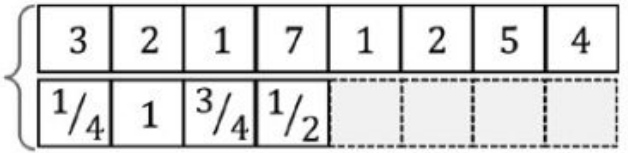
Step 1: Rotate the filter

Step 2: For each output element i, compute the dot-product $x[i:i+4].w^r$

(move the filter two cells)

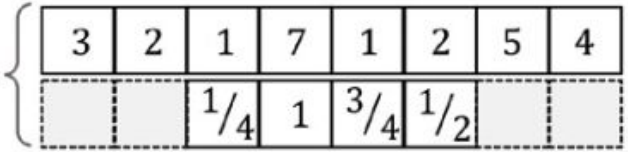
$$y[0] = 3\times{}^1/_4 + 2\times1 + 1\times{}^3/_4 + 7\times{}^1/_2$$

$$\rightarrow y[0] = 7$$



$$y[1] = 1\times{}^1/_4 + 7\times1 + 1\times{}^3/_4 + 2\times{}^1/_2$$

$$\rightarrow y[1] = 9$$



$$y[2] = 1\times{}^1/_4 + 2\times1 + 5\times{}^3/_4 + 4\times{}^1/_2$$

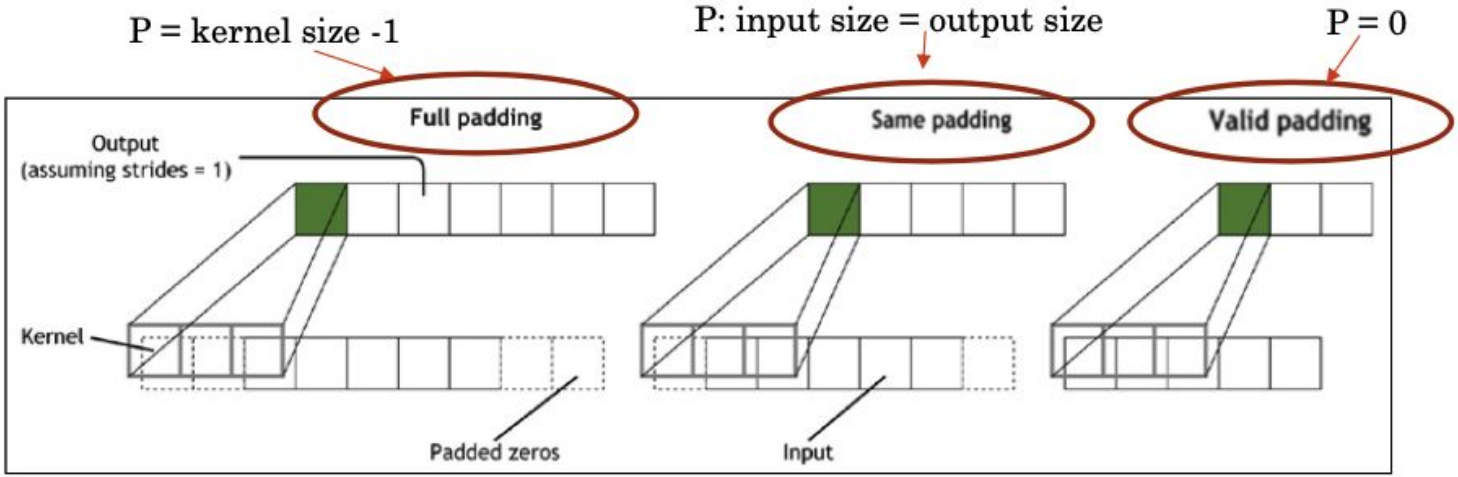
$$\rightarrow y[2] = 8$$



Padding layer

- The result of this convolution is a tensor with a smaller shape than the input one

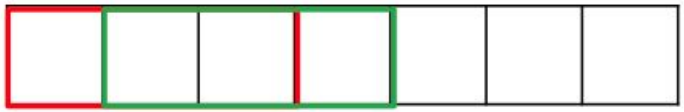
- To preserve/increase shape a **padding procedure** can be applied
  - It consists in padding zero pixels to input tensor
  - Usually, a **same padding** procedure is used, meaning that the output vector has the same size as the input one.



Moving along the input: **Strides**

- One concept introduced in the previous example is the **number of cells the filter is moved when shifted** across the vector x (to pass from a y index to another)

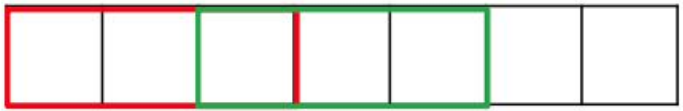
- It is called strides
- **Example: N=7, filter=3**



Stride = 1



Output = 5



Stride = 2



Output = 3

The output size

- The size of the vector obtained by a convolution can be calculated as follows:

$$o = \left[\frac{n + 2p - m}{s}\right] + 1$$

- **o = output dimension**
- n = input dimension
- p = padding
- m = kernel size
- s = stride

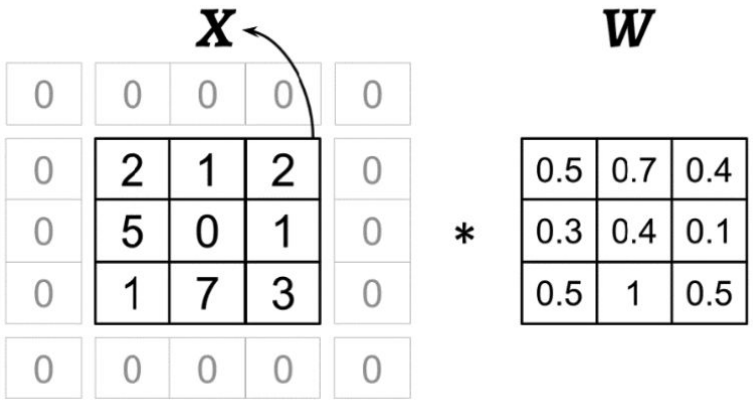
Convolution in 2D

- Extend what we said in 2D:

$$\boldsymbol{Y} = \boldsymbol{X} * \boldsymbol{W} \rightarrow \quad Y[i,j] = \sum_{k_1=-\infty}^{+\infty} \sum_{k_2=-\infty}^{+\infty} X[i-k_1, j-k_2]\, W[k_1, k_2]$$

- $\boldsymbol{X}_{n1\times n2}$ and $\boldsymbol{W}_{m1\times m2}$ are now two matrices → Y is a **2D matrix** as well

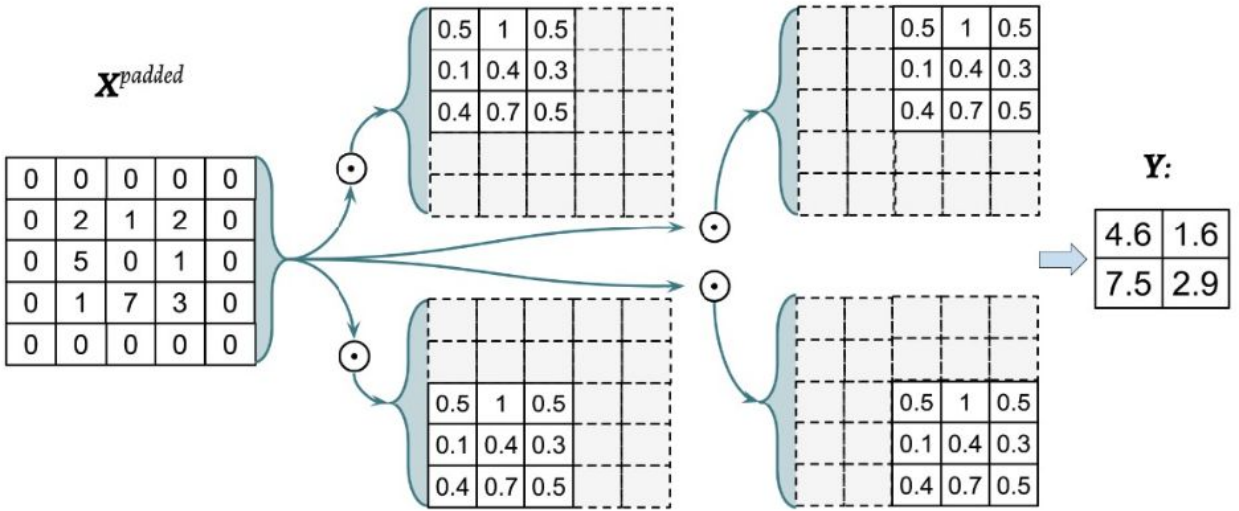
Example:
- input matrix $\boldsymbol{X}_{3\times3}$
- kernel matrix $\boldsymbol{W}_{3\times3}$
- p=(1, 1)
- stride s=(2, 2)



Convolution in 2D

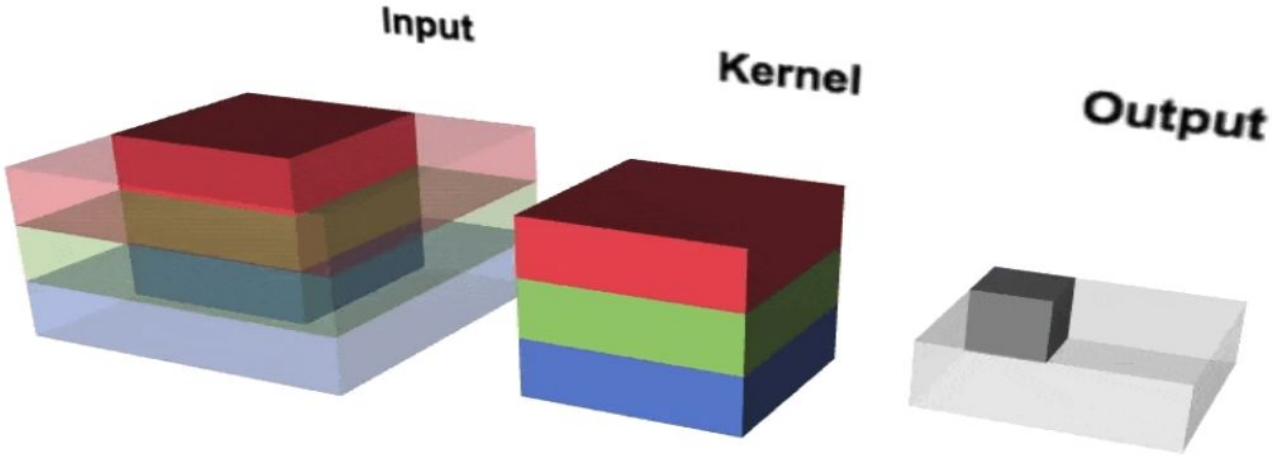
- **Rotate the filter** to perform the sum on indices running in the same directions

$$W^r = \begin{bmatrix} 0.5 & 1 & 0.5 \\ 0.1 & 0.4 & 0.3 \\ 0.4 & 0.7 & 0.5 \end{bmatrix}$$



How does a convolutional layer work on a **RGB image**?

- For each channel color there is a different filter
- The three outputs are added together
- **The output of a convolutional layer with a multi-layer input is a single layer**



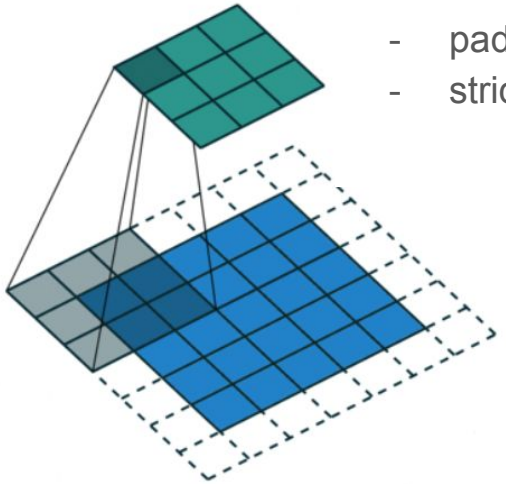
Strides and padding in 2D

- Zero-padding and strides concepts are the same of 1D case
- The output size of a 2D filter is still calculable with the formula seen before, applied on width and height separately

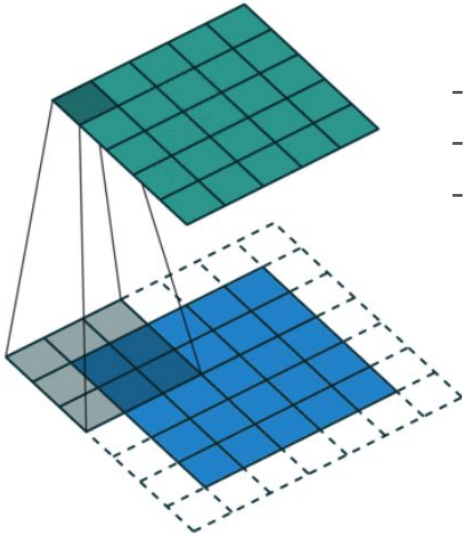
Input dimension Zero-padding Kernel dimension

$$o = \left[\frac{n + 2p - m}{s}\right] + 1$$

stride



- padding (1,1)
- strides (2,2)

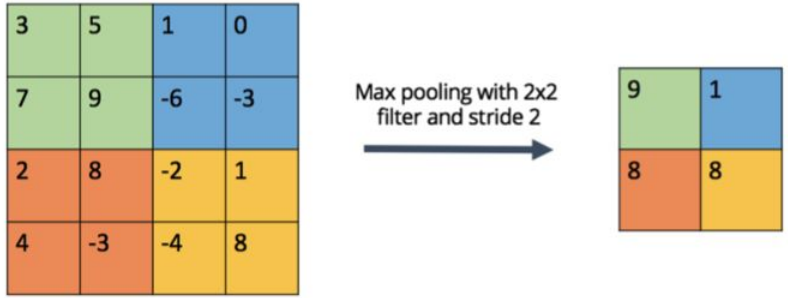


- padding (1,1)
- strides 1
- The input shape is preserved (padding 'same')

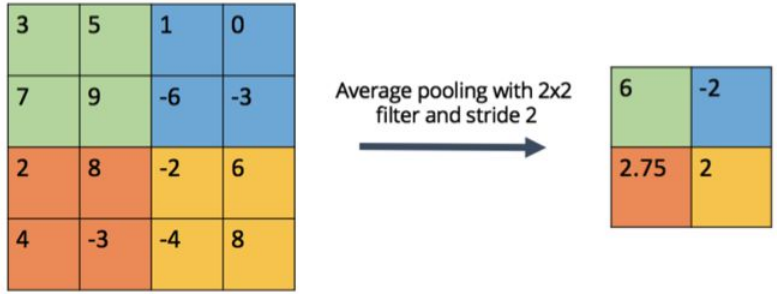
The **pooling layer**

- According to zero-padding and strides it is possible to change (usually **reduce) the input dimension**
- This task can be also performed with a **"Pooling" layer**

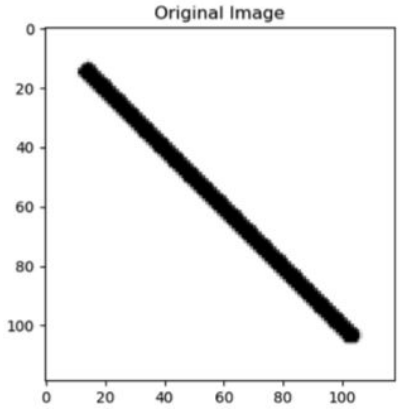
- Two kinds of pooling:
  - **Maximum Pooling** (or Max Pooling): Calculate the maximum value for each patch of the feature map.

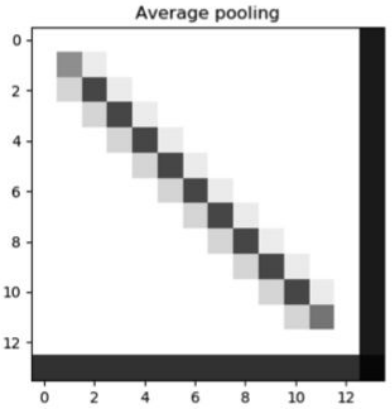


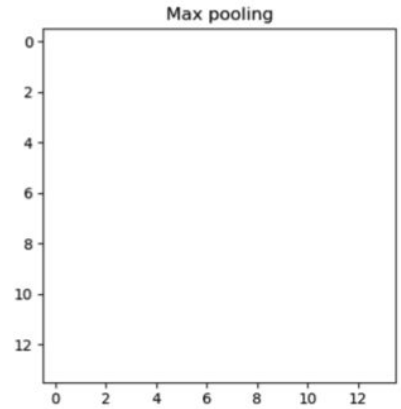
  - **Average Pooling:** Calculate the average value for each patch on the feature map.

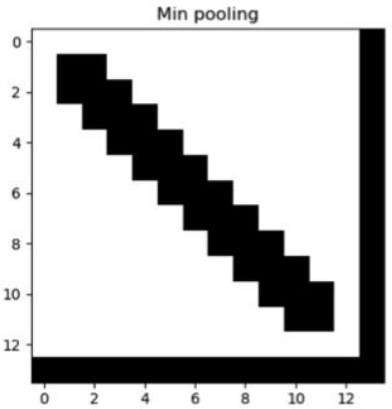


The effect of the pooling layer

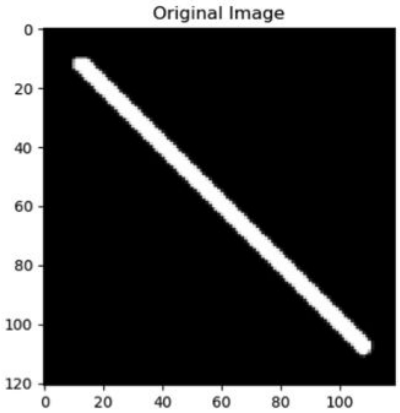


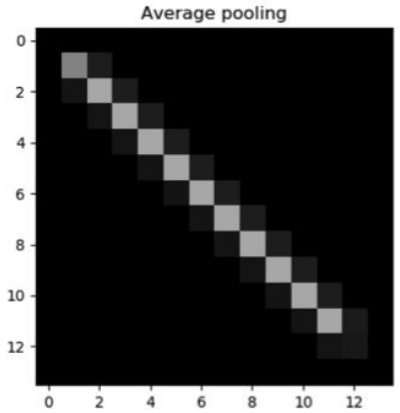


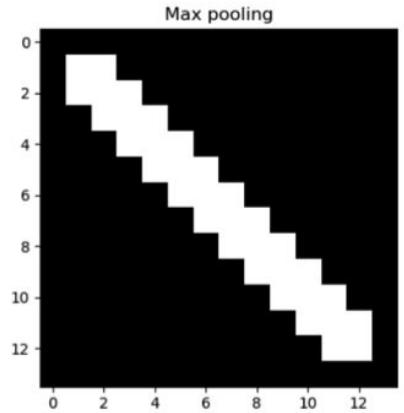


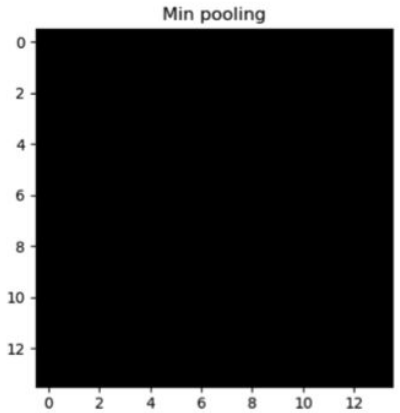


The effect of the pooling layer









Max Pooling

- It basically introduces a **local invariance**
  - **Small changes** in a local neighbourhood **do not change the result of max-pooling**

$$X_1 = \begin{bmatrix} 10 & 255 & 125 & 0 & 170 & 100 \\ 70 & 255 & 105 & 25 & 25 & 70 \\ 255 & 0 & 150 & 0 & 10 & 10 \\ 0 & 255 & 10 & 10 & 150 & 20 \\ 70 & 15 & 200 & 100 & 95 & 0 \\ 35 & 25 & 100 & 20 & 0 & 60 \end{bmatrix}$$

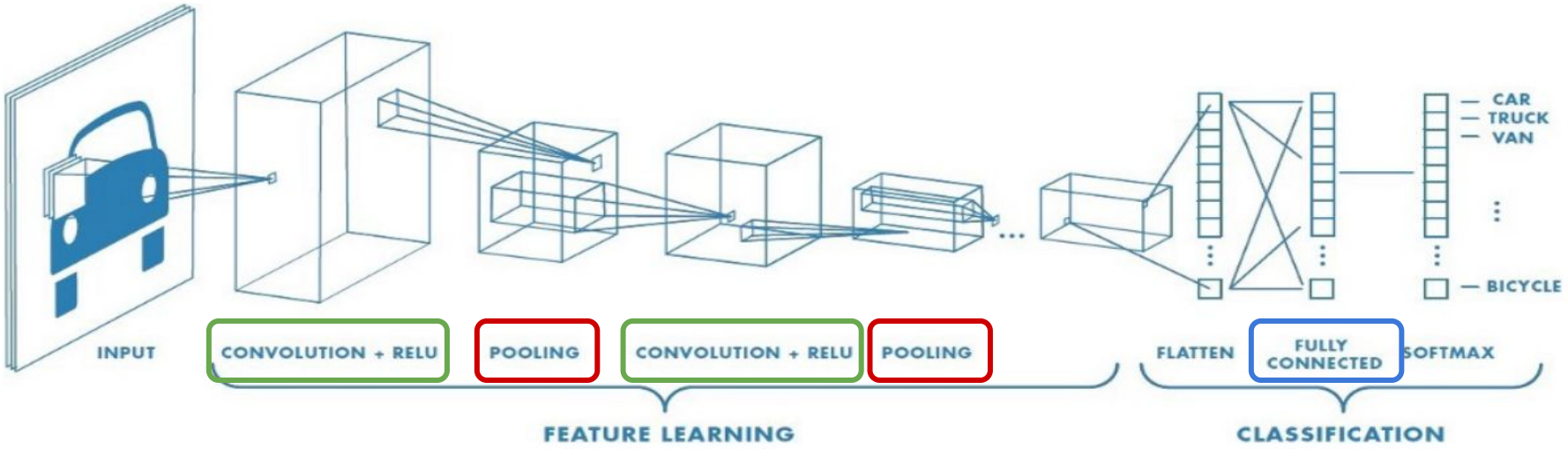
$$X_2 = \begin{bmatrix} 100 & 100 & 100 & 50 & 100 & 50 \\ 95 & 255 & 100 & 125 & 125 & 170 \\ 80 & 40 & 10 & 10 & 125 & 150 \\ 255 & 30 & 150 & 20 & 120 & 125 \\ 30 & 30 & 150 & 100 & 70 & 70 \\ 70 & 30 & 100 & 200 & 70 & 95 \end{bmatrix}$$

$$\xrightarrow{\text{max pooling } P_{2\times 2}} \begin{bmatrix} 255 & 125 & 170 \\ 255 & 150 & 150 \\ 70 & 200 & 95 \end{bmatrix}$$

- **Why we use it?**
  - Pooling decreases the size of features → **higher efficiency**
  - Reduces the risk of overfitting

- Typically, pooling is assumed to be non-overlapping (pooling size = stride)

The big picture



Take away messages

- Filters weights are **learnt from data during training**
- The network learns which are the **most discriminant patterns**
- A cnn performs the **classification** by reading these **extracted features**
- On the contrary, a **dnn** reads only **pixels values**

Different loss functions for different classification problems

- Depending on the type of problem and the type of output, we should choose the **appropriate loss function to train our model**

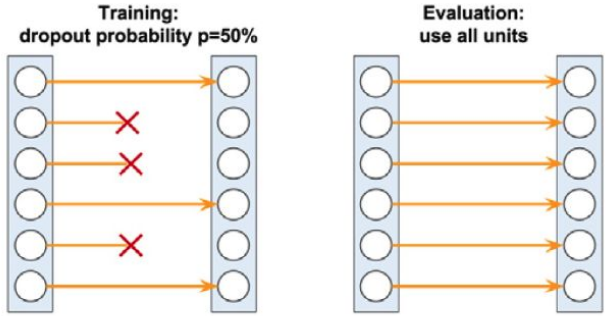
Loss function	Usage	Examples	
		Using probabilities	Using logits
		from_logits=False	*from_logits=True*
`BinaryCrossentropy`	Binary classification	y_true: 1 y_pred: 0.69	y_true: 1 y_pred: 0.8
`CategoricalCrossentropy`	Multiclass classification	y_true: 0 0 1 y_pred: 0.30 0.15 0.55	y_true: 0 0 1 y_pred: 1.5 0.8 2.1
`Sparse CategoricalCrossentropy`	Multiclass classification	y_true: 2 y_pred: 0.30 0.15 0.55	y_true: 2 y_pred: 1.5 0.8 2.1

- **Logit**: sometimes preferred due to numerical stability reasons

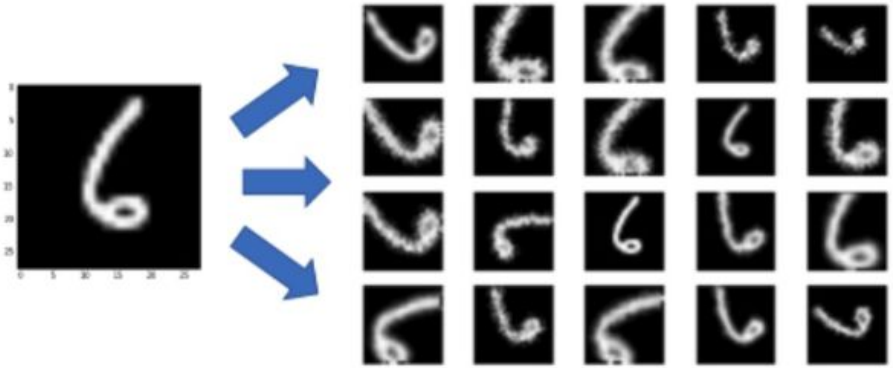
$$\text{logit}(p) = \ln\left(\frac{p}{1-p}\right)$$

DNN regularization argument apply also for CNN

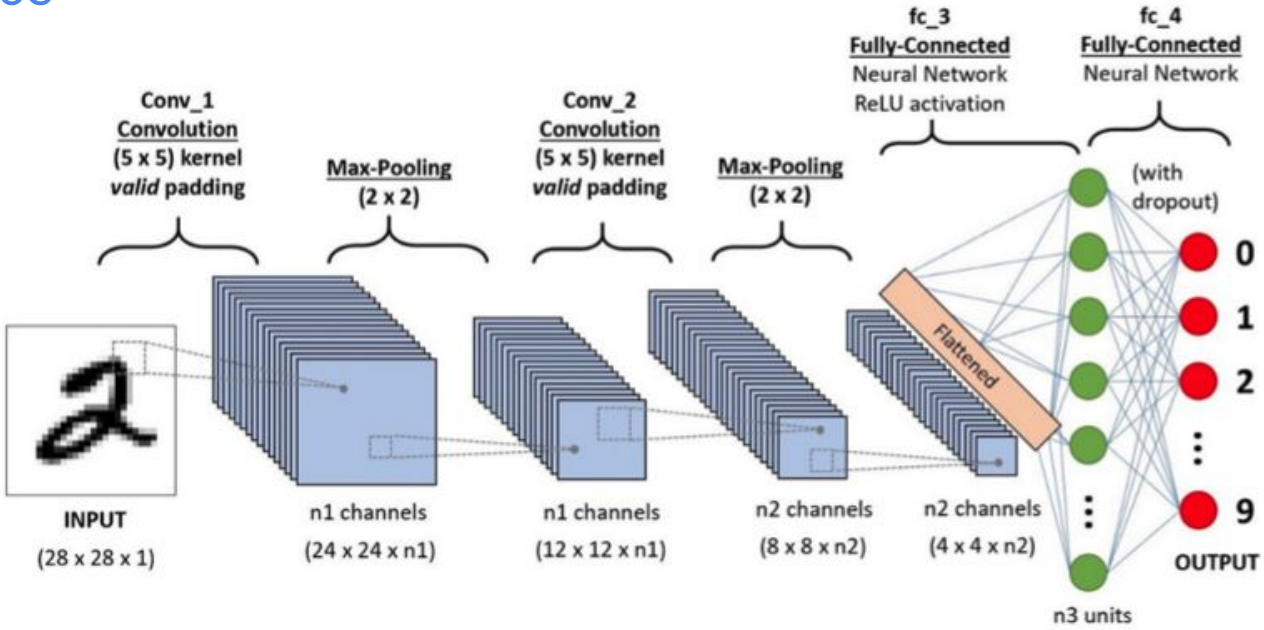
- **Dropout:** to avoid overfitting



- **Data augmentation:** invariance to translation



Good practice



- Since **the first convolutional filters** learn high level features in the image in input and the input size is larger than in inner layers, the **number of filters is relatively small** to not insert too many weights
- A good practice is **to increment this number in the subsequent convolutional steps**
- **Dropout can be inserted not only between dense layers but also between a Conv layer and its input**

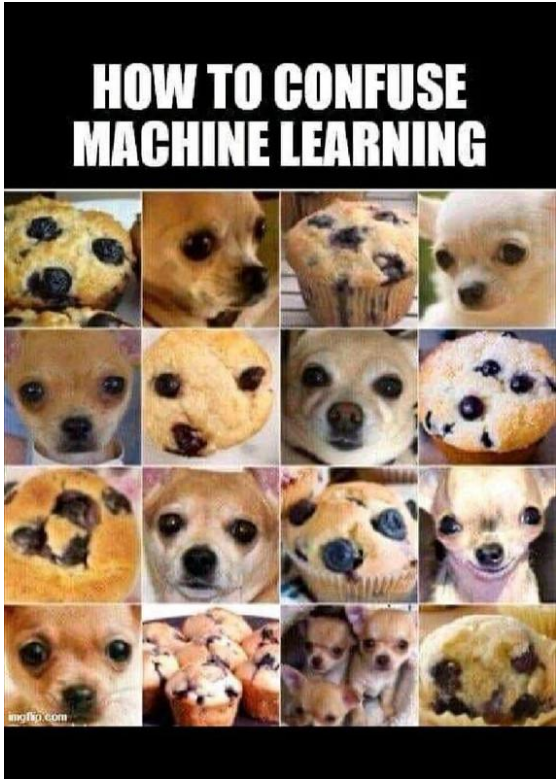
Example…

Hands on example: cats vs dogs

- A dataset containing photos of dogs and cats is provided
  - Pre-processing the images
  - Creating a model
  - Training and testing
  - Evaluating performances
  - Improving the model!
- Have a look at this notebook built to analyze images from this dataset!



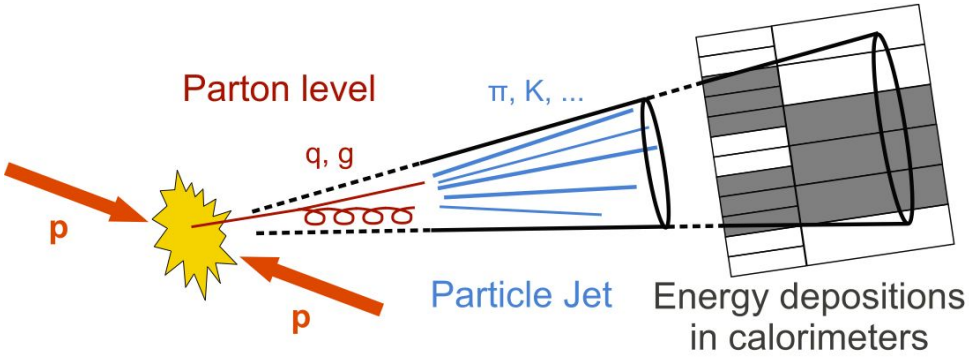
A more challenging exercise: dogs vs muffins ;)

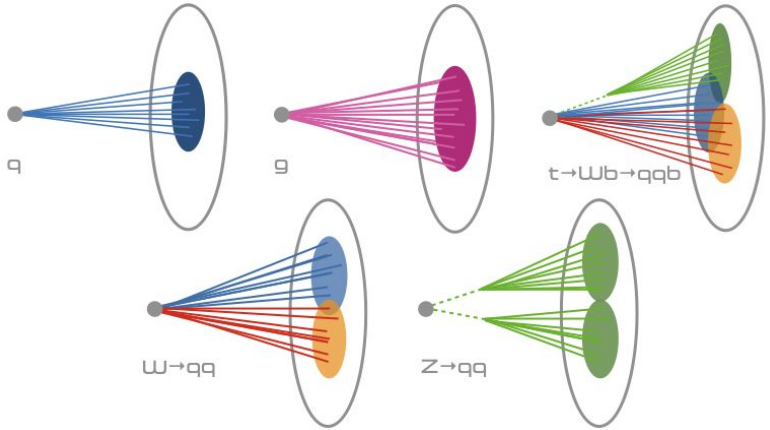


Hands on…

Classification of particle jets - multiclass classification

- **Jet tagging** at the Large Hadron Collider is a critical data analysis technique used to identify the originating particle of a jet
  - A jet is a collimated spray of particles produced in high-energy collisions
- Except for quark- (“q”) and gluon-initiated jets (“g”), all other jets in the dataset have multiple "prongs"
  - The analysis of the structure of the jet provides info on the jet type
- → **Implement a ML approach that uses CNN to perform multiclass classification**



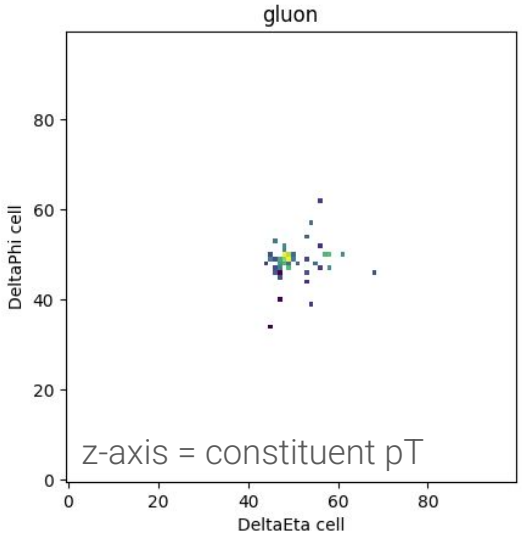


Dataset composition

- The dataset can be found here in the form of a `.npz` file
- A notebook showing how to access the variables is available here instead

- 20k events (jets) in total
  - Each jet has up to 100 jet constituents (if smaller than 100 is zero-padded to 100)
  - 16 features per constituent including geometrical variables such as eta/phi

- The idea is to **classify jets based on the 2D histogram of the transverse momentum (pT) deposited in each of 100x100 bins of a square window of the (η, ϕ) plane**

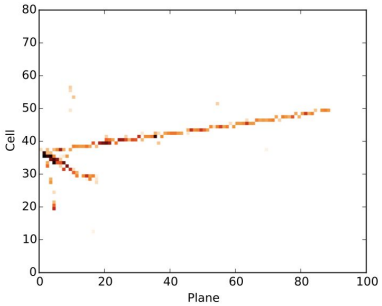


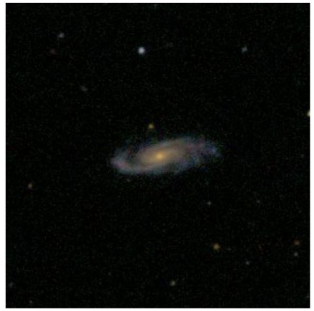
Typical jet image in the eta/phi plane

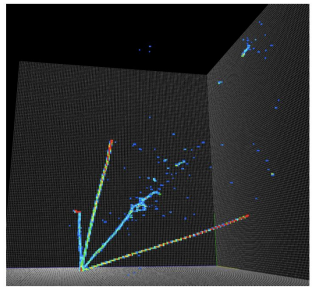
100x100 pixels

One word on **data sparsity**

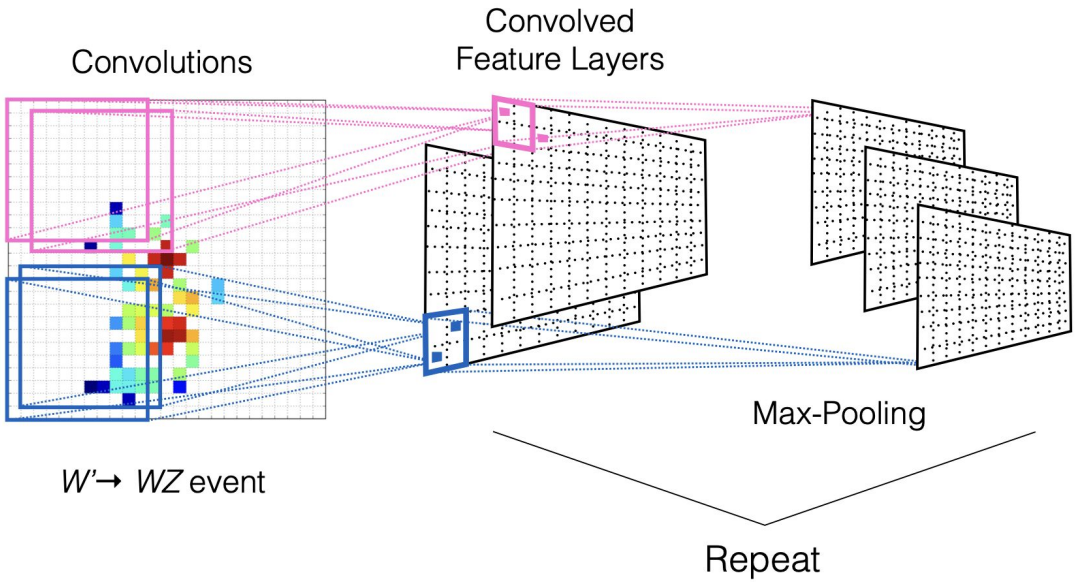
- In physics, image data is often sparse (mostly empty)
  - Even worse for 3D data
- How can we deal with this efficiently?







Use large filters!



3x3 filters are common for natural images, use larger filters (e.g. 11x11) for sparse images